

JCST Papers

Only for Academic and Non-Commercial Use

Thanks for Reading!



[Survey](#)

[Computer Architecture and Systems](#)

[Artificial Intelligence and Pattern Recognition](#)

[Computer Graphics and Multimedia](#)

[Data Management and Data Mining](#)

[Software Systems](#)

[Computer Networks and Distributed Computing](#)

[Theory and Algorithms](#)

[Emerging Areas](#)



JCST WeChat

Subscription Account

JCST URL: <https://jcast.ict.ac.cn>

SPRINGER URL: <https://www.springer.com/journal/11390>

E-mail: jcast@ict.ac.cn

Online Submission: <https://mc03.manuscriptcentral.com/jcast>

Twitter: JCST_Journal

LinkedIn: Journal of Computer Science and Technology

AI Computing Systems for Large Language Models Training

Zhen-Xing Zhang^{1, 2} (张振兴), Yuan-Bo Wen² (文渊博), Han-Qi Lyu^{1, 2, 3} (吕涵祺), Chang Liu³ (刘畅)
Rui Zhang² (张蕊), Xia-Qing Li² (李夏青), Chao Wang¹ (王超), *Senior Member, CCF*
Zi-Dong Du^{2, 4} (杜子东), *Senior Member, CCF*, Qi Guo² (郭崎), *Senior Member, CCF*
Ling Li⁵ (李玲), *Distinguished Member, CCF*, Xue-Hai Zhou¹ (周学海), *Senior Member, CCF*
and Yun-Ji Chen^{2, 6, *} (陈云霁), *Fellow, CCF*

¹ School of Computer Science and Technology, University of Science and Technology of China, Hefei 230026, China

² State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing 100190, China

³ Cambricon Technologies, Beijing 100191, China

⁴ Shanghai Innovation Center for Processor Technologies, Shanghai 201210, China

⁵ Intelligent Software Research Center, Institute of Software, Chinese Academy of Sciences, Beijing 100190, China

⁶ University of Chinese Academy of Sciences, Beijing 101408, China

E-mail: winkzy@mail.ustc.edu.cn; wenyuanbo@ict.ac.cn; lvhanqi@mail.ustc.edu.cn; liuchang1@cambricon.com
zhangrui@ict.ac.cn; lixiaqing@ict.ac.cn; cswang@ustc.edu.cn; duzidong@ict.ac.cn; guoqi@ict.ac.cn; liling@iscas.ac.cn
xhzhou@ustc.edu.cn; cyj@ict.ac.cn

Received February 8, 2024; accepted January 5, 2025.

Abstract In this paper, we present a comprehensive overview of artificial intelligence (AI) computing systems for large language models (LLMs) training. The rapid advancement of LLMs in recent years, coupled with the widespread adoption of algorithms and applications such as BERT, ChatGPT, and DeepSeek, has sparked significant interest in this field. We classify LLMs into encoder-only, encoder-decoder, and decoder-only models, and briefly analyze their training and inference processes to emphasize their substantial need for computational resources. These operations depend heavily on AI-specific accelerators like GPUs (graphics processing units), TPUs (tensor processing units), and MLUs (machine learning units). However, as the gap widens between the increasing complexity of LLMs and the current capabilities of accelerators, it becomes essential to adopt heterogeneous computing systems optimized for distributed environments to manage the growing computational and memory requirements of LLMs. We delve into the execution and scheduling of LLM algorithms, underlining the critical role of distributed computing strategies, memory management enhancements, and boosting computational efficiency. This paper clarifies the complex relationship between algorithm design, hardware infrastructure, and software optimization, and provides an in-depth understanding of both the software and hardware infrastructure supporting LLMs training, offering insights into the challenges and potential avenues for future development and deployment.

Keywords artificial intelligence (AI) chip, large language model (LLM), AI computing system, accelerator

1 Introduction

Large language models (LLMs) have become a key area of focus within the realms of artificial intelligence (AI) with significant achievements across various applications such as machine translation, senti-

ment analysis, question-answering systems, and text generation. The evolution of LLMs, grounded in deep learning, has been particularly accelerated since the introduction of the Transformer network by Google in 2017^[1], which featured attention mechanisms. Subsequent studies, including the BERT^[2], GPT series^[3, 4],

Survey

This work was partially supported by the National Natural Science Foundation of China under Grant Nos. 61925208, U22A2028, 62302483, 62222214, 62341411, 62102399, and 62372436, the Chinese Academy of Sciences (CAS) Project for Young Scientists in Basic Research under Grant No. YSBR-029, the Youth Innovation Promotion Association of CAS, and Xplore Prize.

*Corresponding Author

©Institute of Computing Technology, Chinese Academy of Sciences 2025

and DeepSeek series^[5, 6] work, have pushed the development of LLMs, especially with the launch of ChatGPT^① at the end of 2022, sparking a new surge in research on generalized LLMs.

The rapid development of LLMs is heavily linked to the AI computing systems supporting their computation. Modern AI computing systems typically combine general-purpose CPUs with heterogeneous accelerators equipped with AI chips, complemented by corresponding programming environments that include both frameworks and languages^[7]. The demand for AI computing is growing exponentially, while the development of CPUs has stagnated, positioning high-performance AI chips at the center of modern AI computing systems. Since the introduction of the pioneering deep learning processor architectures, DianNao^[8] and its subsequent DianNao family series^[9–11], there has been a notable decrease in computational costs and an increase in processing efficiency. Accelerators such as NVIDIA’s TensorCore GPU (graphics processing unit), Google’s TPU^② (tensor processing unit)^[12], and Cambricon’s MLU (machine learning unit) have further enhanced the performance advantage over traditional CPUs, driving rapid advances in the field of LLMs and AI computing system.

With the advancing computational capabilities of AI computing systems, the scope and size of deep learning models, particularly LLMs, have seen unprecedented growth. This rapid expansion outpaces the advancements in AI computing capabilities, high-

lighting the urgent need to distribute work across multiple accelerators and employ distributed systems for collaborative training. To manage the substantial parameter volumes and extensive datasets inherent to these models, AI computing systems for LLMs should be optimized for distributed environments. As shown in Fig.1, starting from the top, large language applications depend on specific models, which rely on various programming frameworks for execution. Specifically, the operation of LLMs depends on advanced deep learning frameworks, such as DeepSpeed^[13–16], Megatron-LM^[17, 18], FSDP (Fully Sharded Data Parallel)^③, which further invoke lower-level deep learning programming frameworks, primarily PyTorch. These low-level frameworks leverage high-performance computing libraries, communication libraries, and runtime systems to perform complex computational tasks and control the transmission across the communication network.

There are a few related surveys in the research^[19–25]. Some primarily examine LLM algorithms but do not adequately distill and analyze underlying computational principles; some address LLM inference deployment without discussing how training and inference differ, whether algorithmically, in software implementation, or hardware deployment; and others are devoted to distributed training of DNNs (deep neural networks), without a specific emphasis on LLMs.

However, to the best of our knowledge, there is currently no work that systematically reviews the

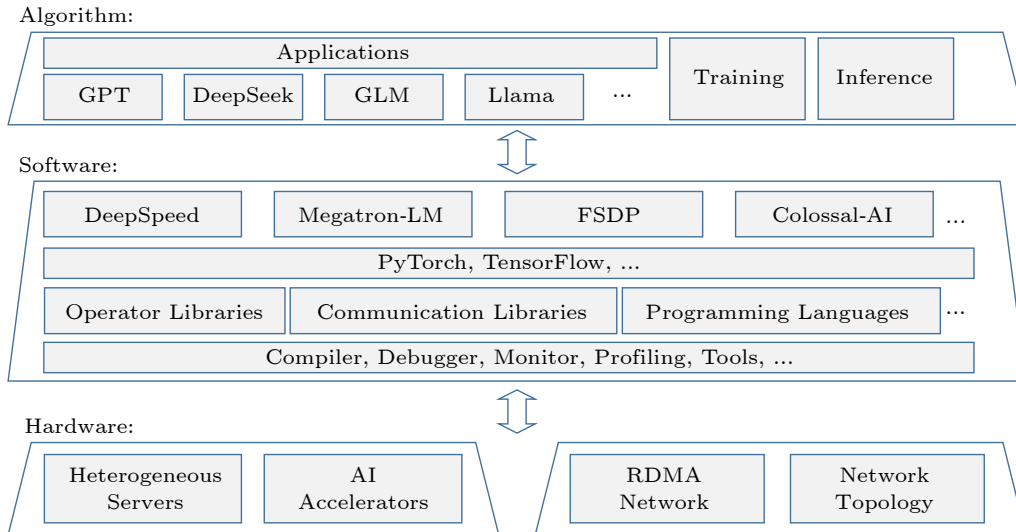


Fig.1. Overall architecture of AI computing systems for LLMs.

①<https://openai.com/index/chatgpt/>, Jan. 2025.

②<https://cloud.google.com/tpu>, Jan. 2025.

③<https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/>, Jan. 2025.

computing systems for LLMs. By connecting top-level LLMs algorithms with mid-level programming software and base-level intelligent chips and accelerators, we aim to help readers develop a systematic understanding of AI's entire software and hardware stack.

From a systematic viewpoint, evaluating AI-related work encompasses more than accuracy; other factors like throughput, latency, energy efficiency, and cost are also important. Advances in any of these areas are valuable contributions.

In this paper, we review existing algorithms for LLMs as well as the hardware platforms on which they run, and then introduce the distributed scheduling strategies for deploying algorithms onto hardware platforms, along with corresponding optimizations. Our taxonomy is shown in Fig.2. First, we classify the mainstream LLMs, identify their common computational blocks and features, and discuss two execution modes: training and inference. Then, we categorize the hardware platforms for running LLMs, discussing

computing nodes and computing clusters separately, with the goal of guiding hardware infrastructure selection and addressing the main communication bottlenecks and challenges. Consecutively, our discussion extends to the execution and scheduling of LLM algorithms on hardware platforms, exploring software strategies for optimization. We review LLM programming frameworks, and then detail optimization in distributed computing, memory management, and computational efficiency, as well as comparing training and inference techniques. Finally, we outline future work in terms of algorithm, hardware, and software.

To the best of our knowledge, this paper is the first work to survey LLMs from a system-level perspective. This survey contributes as follows.

1) We briefly introduce the concept of AI computing systems to provide readers with an in-depth understanding of the software and hardware infrastructure that supports LLMs training.

2) We analyze workflows, computational process-

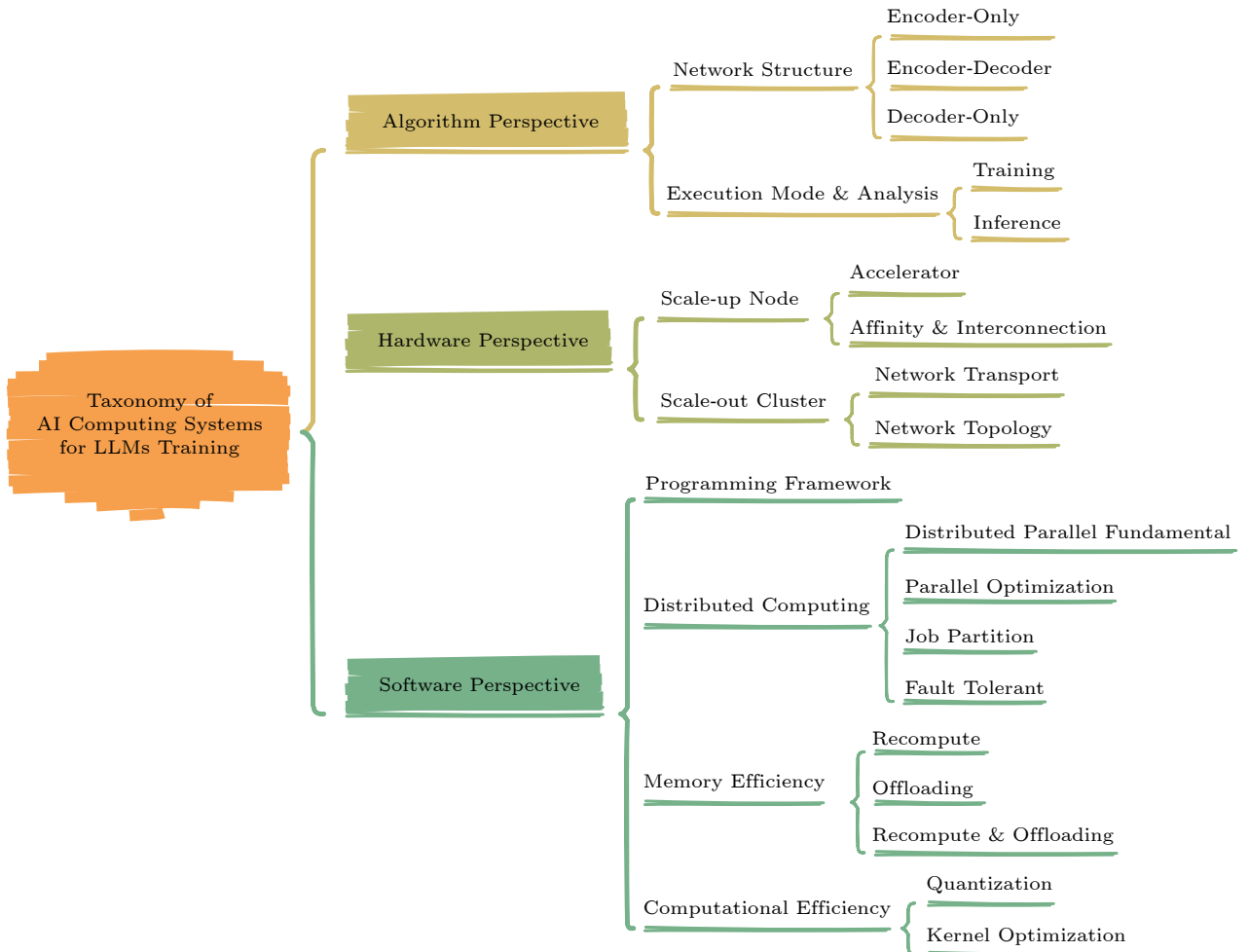


Fig.2. Taxonomy of AI computing systems for LLMs.

es, and communication patterns for distributed LLMs training and corresponding optimization techniques.

3) We highlight three key challenges and offer insights into the potential avenues for future development in AI computing systems for LLMs training.

The rest of this paper is organized as follows. In [Section 2](#), we discuss the network structure and execution mode of LLM algorithms, and provide a brief analysis. [Section 3](#) is dedicated to presenting the platforms utilized for running LLMs, and it explores the crucial elements influencing both scale-up and scale-out. [Section 4](#) explores software strategies for deploying and optimizing LLM algorithms on hardware, covering programming frameworks, distributed computing, memory management, and computational enhancements, alongside a brief comparison with inference. [Section 5](#) is about challenges and future work. [Section 6](#) concludes the paper.

2 LLM Algorithms

This section outlines LLMs from an algorithmic perspective, focusing on their structure types: encoder-only, encoder-decoder, and decoder-only. It then explores the computational workflow for LLMs, including training (pre-training and fine-tuning) and inference. The discussion concludes with an analysis of the operational intensity, highlighting the critical role of optimizing the attention mechanism in the Transformer^[1] structure to improve computational efficiency.

2.1 Network Structure

Pretrained LLMs are typically based on the Transformer^[1] architecture, yet they are not identical to it. The Transformer architecture is crucial in natural language processing (NLP), particularly for its effective handling of long-range dependencies and parallel processing capabilities. Originally conceived for the domain of translation, this structure is comprised of encoder and decoder components. Encoders leverage self-attention mechanisms to process incoming text, extracting pertinent information and generating hidden states that encapsulate the essential characteristics of the input. Decoders, employing these hidden states and cross-attention mechanisms, proceed to sequentially produce translated text. For LLMs, the Transformer architecture has been expanded and optimized to accommodate a wider range of complex tasks^[20, 21], such as text generation, summarization,

and question answering, thus extending beyond its initial scope. These enhancements typically involve augmenting the performance and adaptability of LLMs through an increased layer count, expanding the number of parameters, scaling up the training datasets, and advancing training techniques and optimization approaches.

As shown in [Fig.3](#), LLM structures based on Transformer can be broadly categorized into three types: encoder-only, encoder-decoder, and decoder-only structures, each serving distinct purposes in the realm of NLP.

1) The encoder-only structure stacks multiple encoder blocks and adds different output layers for various tasks. A typical example is BERT^[2], which employs bidirectional attention to grasp the context of language. It excels in natural language understanding, but is not inherently suited for NLP tasks.

2) The encoder-decoder structure adds decoder blocks after the encoder blocks for sequential outputs. It uses the output of the last encoder block as the input of the cross-attention in decoder blocks, fully embracing and enhancing the Transformer's design. Google's T5^[26] is a notable example, using encoders for contextual understanding and decoders for sequential, autoregressive output generation. The downside is their larger parameter size and lower computational efficiency.

3) The decoder-only structure omits the encoder and stacks multiple decoder blocks, focusing on natural language generation. A typical example is the GPT series^[3, 4, 27], which excels in generation tasks and addresses understanding tasks through extensive pretraining. This demonstrates its versatility in applications, making the decoder-only structure a popular choice for LLMs. Despite their similar appearance to the structure of the encoder-only model in [Fig.3](#), decoder-only models function differently in practice, utilizing an autoregressive method, which is discussed in [Subsection 2.2](#).

[Table 1](#) presents a compilation of notable LLMs and their relevant details, including their network structure types. The development of LLMs saw a significant advancement in 2017 with Google's introduction of the attention mechanism and the Transformer architecture^[1]. This innovation involves training an encoder-decoder model based on this structure with 210 million parameters, which serves as the milestone for subsequent work in the field of LLMs. OpenAI released GPT-2^[3] in 2019 and GPT-3^[4] in 2020, both

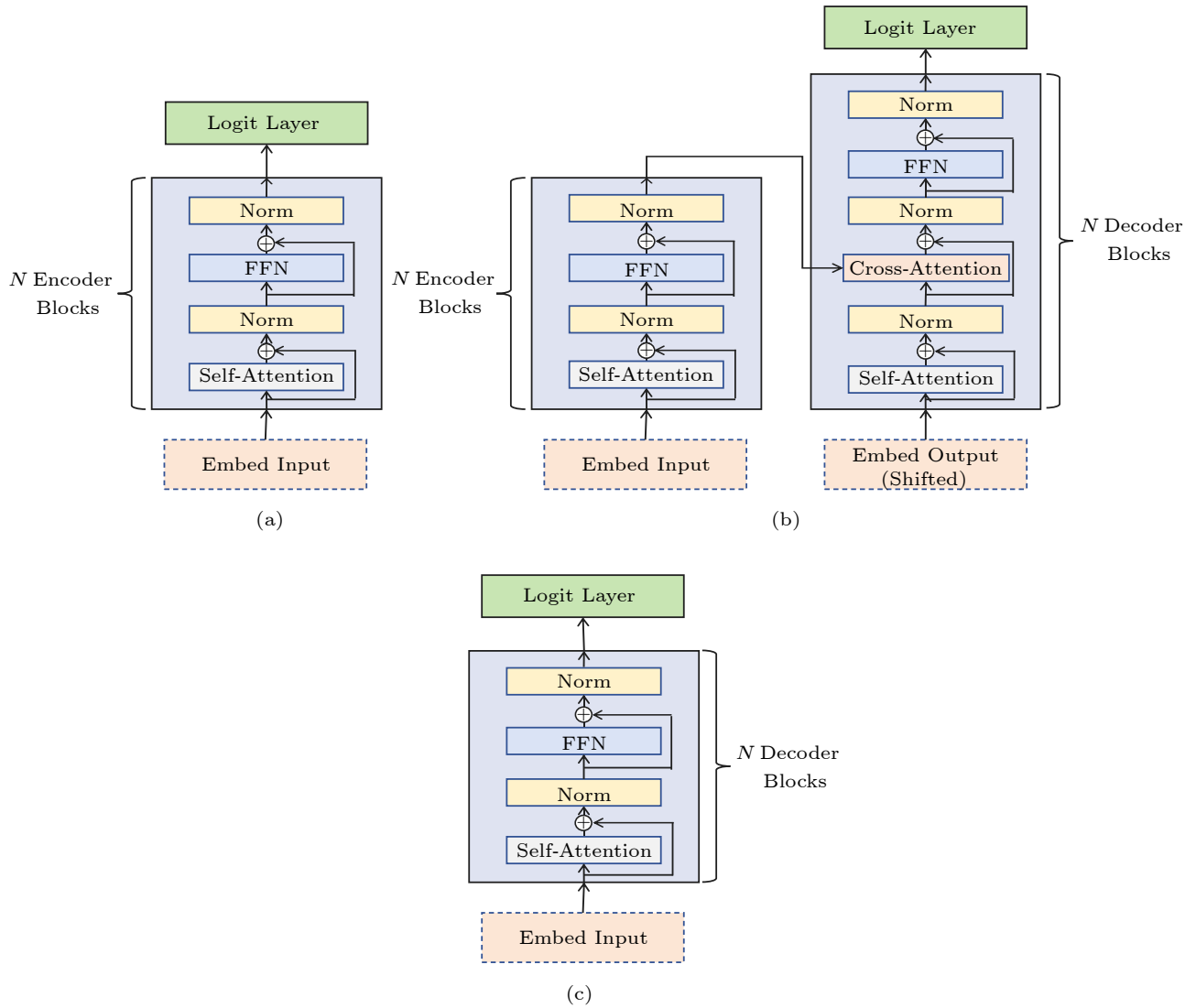


Fig.3. Comparison of network structures. (a) An encoder-only model with N blocks. (b) An encoder-decoder model with $2N$ blocks. (c) A decoder-only model with N blocks. Note: the last layer (logit layer) varies depending on the final application of the model.

based on the Transformer architecture. GPT-2 has over 1 billion parameters, and GPT-3 has hundreds of billions of parameters, pretrained on around 300 billion tokens. Competition has intensified as Baidu launched Ernie^[28], Google released PaLM^[30] and Gemini^[38], Alibaba introduced QWen^[39], and Meta proposed the Llama models^[34, 36], which process larger datasets with smaller models for better results with equivalent parameters. In 2024, Meta launched Llama 3 models^[4, 5], the largest version of which has 405 billion parameters. Unfortunately, the disclosure of technical details in recent LLM developments has significantly decreased. However, the release of DeepSeek V3^[6] has breathed new life into the open-source LLM

community.

In recent years, the escalation in model parameters and the volumes of training data have rendered the training of large-scale models an exceedingly resource-intensive task. The research by DeepMind^[40] indicates that the computational operations for training LLMs are directly proportional to the number of model parameters and the size of the training dataset, highlighting an ongoing surge in computational resource requirements. Furthermore, the mixture of experts (MoE) technique integrates multiple “expert” sub-networks^[41–43], typically in FFN layers. Each expert is a model designed to handle distinct task types or data categories, and is activated for computation

^[4]<https://github.com/meta-llama/llama3>, Jan. 2025.

^[5]<https://ai.meta.com/blog/meta-llama-3-1/>, Jan. 2025.

Table 1. Statistics of Representative Large Language Models

Model	Organization	Release Date	Structure	(l, n, d)	Parameter Size	Dataset Size
Transformer ^[1]	Google	2017.6	Encoder-decoder	(12, 8, 64)	213M	15B tokens
BERT ^[2]	Google	2018.1	Encoder-only	(24, 16, 64)	340M	128B tokens
GPT-2 ^[3]	OpenAI	2019.2	Decoder-only	(48, 25, 64)	1.5B	40 GB
T5 ^[26]	Google	2019.1	Encoder-decoder	(24, 128, 8)	11B	1T tokens
GPT-3 ^[4]	OpenAI	2020.5	Decoder-only	(96, 96, 128)	175B	300B tokens
Ernie 3.0 ^[28]	Baidu	2021.7	Encoder-decoder	–	10B	4 TB
Gopher ^[29]	DeepMind	2021.12	Decoder-only	(80, 128, 128)	280B	300B tokens
PaLM ^[30]	Google	2022.4	Decoder-only	(118, 48, 384)	540B	780B tokens
OPT ^[31]	Meta AI	2022.5	Decoder-only	(96, 96, 128)	175B	180B tokens
BLOOM ^[32]	BigScience	2022.7	Decoder-only	(70, 112, 128)	176B	366B tokens
GLM ^[33]	THU	2022.1	Encoder-decoder	(70, 96, 128)	130B	400B tokens
ChatGPT	OpenAI	2022.11	Decoder-only	–	–	–
LLaMA ^[34]	Meta AI	2023.2	Decoder-only	(80, 64, 128)	65B	1.4T tokens
GPT-4 ^[27]	OpenAI	2023.3	Decoder-only	–	–	–
PanGu ^[35]	Huawei	2023.3	Decoder-only	–	1085B	329B tokens
Llama 2 ^[36]	Meta AI	2023.7	Decoder-only	–	70B	2T tokens
Monkey ^[37]	HUST	2023.11	Decoder-only	–	9.8B	–
Gemini ^[38]	Google	2023.12	Decoder-only	–	–	–
Qwen 1.5 ^[39]	Alibaba	2024.2	Decoder-only	–	72B	–
Llama 3	Meta AI	2024.4	Decoder-only	–	405B	15T tokens
DeepSeek V3 ^[6]	DeepSeek	2024.12	Decoder-only&MOE	–	671B	14.8T tokens

Note: l means the number of layers, n represents the number of attention heads, and d equals the dimension of the key. It should be noted that there is no data from official sources available to illustrate the parameter sizes of models such as ChatGPT, GPT-4, and Gemini (Ultra version).

when needed. This methodology significantly enhances the models’ scalability and flexibility across various NLP tasks, while also facilitating an increase in model size and highlighting the need for a load balancing strategy across the expert sub-networks to ensure both algorithmic effectiveness and computational efficiency. At the same time, the development of LLMs has gone beyond traditional language and text, gradually expanding to integrate multimodal data such as images and speech. Models such as Monkey^[37] are capable of processing visual information, GPT-4o^⑥ showcases impressive speech capabilities, and OpenAI’s Sora^⑦ has the ability to create videos from textual descriptions.

In addition to the growth of model size, significant advancements aimed at enhancing model accuracy and efficiency in algorithms have been progressively proposed. Among these, simplified attention (or efficient attention) is particularly significant.

Simplified attention refers to reducing computing operations by leveraging that “only a select number of key elements are active or significant”, enhancing

computational efficiency via model simplification and pruning techniques. Given the widespread issue of parameter redundancy in deep learning models, exploring the sparsity of the model can lead to a substantial reduction in either the number of computational operations or the overall size of the model parameters.

A notable challenge in LLMs is the quadratic growth in the memory scale and computational operations required for attention mechanisms, which will be explained in [Subsection 2.3](#), with a complexity of $O(s^2)$ (s for sequence length). This quadratic growth becomes particularly problematic when the attention for ultra-long sequences is computed. The introduction of simplified attention mechanisms (such as strided “sparse attention” in Sparse Transformer^[44], “hashing attention” in Reformer^[45], “sliding attention” in Longformer^[46], “random attention” in Big Bird^[47], “finite length attention” in StreamingLLM^[48], “attention mask” in PCW^[49], and “dilated attention” in LONGNET^[50]) can effectively simplify the computational process. These mechanisms are instrumental in processing long texts, making improvements in

⑥<https://openai.com/index/hello-gpt-4o/>, Jan. 2025.

⑦<https://openai.com/index/video-generation-models-as-world-simulators/>, Jan. 2025.

computational and memory efficiency^[51].

In addition, a number of algorithmic improvements have been proposed in terms of layer normalization, positional coding, and activation functions^⑧.

- Layer normalization is a technique pivotal for enhancing accuracy and training converge stability by normalizing the activations of each layer. It is now widely used in various forms including LayerNorm, RMSNorm, DeepNorm, and more.

- Position encoding is essential for incorporating the relative or absolute positional information of current word embeddings within a sequence into Transformer models, and now features a range of methods such as absolute position embedding, relative position embedding, Attention with Linear Biases, and Rotary Position Embedding (RoPE).

- Activation functions are used to provide nonlinear features to neural networks. Traditional methods like ReLU and GeLU have been complemented by new ones such as ReGLU, GEGLU, and SwiGLU.

These advancements underline the constant evolution of the LLM landscape, even though their adjustments are localized. Encoder-only, decoder-only, and encoder-decoder models, each with unique strengths and limitations, cater to diverse applications. While decoder-only models are prevalent, encoder-only and encoder-decoder models continue to advance, showcasing their potential in specific areas^[52-56]. This diversification reflects the evolving landscape of LLMs and their adaptation to various linguistic and computational challenges.

2.2 Execution Mode

The computation of LLMs mainly involves two different types: training and inference. The training phase is further divided into pre-training and fine-tuning stages, with fine-tuning described in work like LoRA^[57], Prefix Tuning^[58], and Adapter^[59]. Both extensively utilize substantial datasets and considerable computational resources. Pre-training involves using general corpus data to train the model from scratch, whereas fine-tuning employs domain-specific data to refine the model's weights. Despite the divergence in data sources, the underlying computational paradigms are the same. Additionally, advanced techniques such as Instruction Fine-Tuning (IFT)^[60], Chain of Thought processing (CoT)^[61], and Reinforcement

Learning from Human Feedback (RLHF)^[62] are essential for improving the practical application of LLMs. These techniques significantly enhance the quality of the model's output and interaction capabilities by accurately guiding the model with precise instructions, breaking down complex problems into interpretable steps, and incorporating feedback from human experts.

The training involves steps like forward and backward propagation, along with updating parameters. Data flows through the network topology during forward propagation, while backward propagation necessitates the reverse flow of gradient information, with parameter adjustments through various optimization algorithms. Most training of these models employs synchronous methods, which have the advantage of maintaining stability and consistency during the training process, but this also brings a significant computational burden. For instance, the training phase requires additional memory for the optimizer states and gradients. Simultaneously, activation values generated in forward propagation must be retained for subsequent utilization during backward propagation. These memory requirements pose significant challenges to further optimization.

In the training process of LLMs, while specific training strategies and carefully chosen hyperparameter settings^[63] do not alter the basic computational steps of forwarding, backward propagation, and parameter updates, they are crucial for reducing training costs and improving model accuracy. Fundamental initial configurations, such as weight initialization, significantly influence the model's early learning state. Suitable initial learning rates, and learning rate adjustment strategies (e.g., warmup, learning rate delay mechanisms such as LARS, LAMB, cosine delay) are essential for ensuring rapid and stable convergence in large-batch training. Furthermore, the choice of suitable optimizers (like Adam, AdamW, AdaFactor, or SGD), coupled with techniques like weight decay to prevent overfitting, grad clip to avert gradient explosion, and the utilization of dropout methods, have all been proven to improve training efficiency and model stability.

As for inference, it refers to using the trained model to perform specific tasks, which includes only forward propagation, and various optimization strategies might be applied to achieve high throughput or

^⑧For references, please see the supplement "4178_Supplement_document.pdf" at <https://jcst.ict.ac.cn/article/doi/10.1007/s11390-024-4178-1>, Jan. 2025.

low latency. We mainly discuss the training methods and optimizations in this survey.

Taking auto-regressive decoder-only models as an example, it predicts sequences by depending on all preceding tokens for generating new ones. During training, these models apply techniques such as teacher forcing^[64] and attention mask^[1] to facilitate parallel training. Teacher forcing uses the correct token instead of the model's previous output as the input for the next prediction. Attention mask is used to block the information of subsequent tokens, for parallel sequence processing. In the inference phase, the token-by-token computation process involves repeating r iterations, where each iteration relies on its own previous prediction output to generate the next token in the sequence. When utilizing the KV cache, the input is only the word predicted in the previous iteration. This means that the inference process can only proceed sequentially. As shown in Fig.4, inference performs many iterations, and cannot be parallelized.

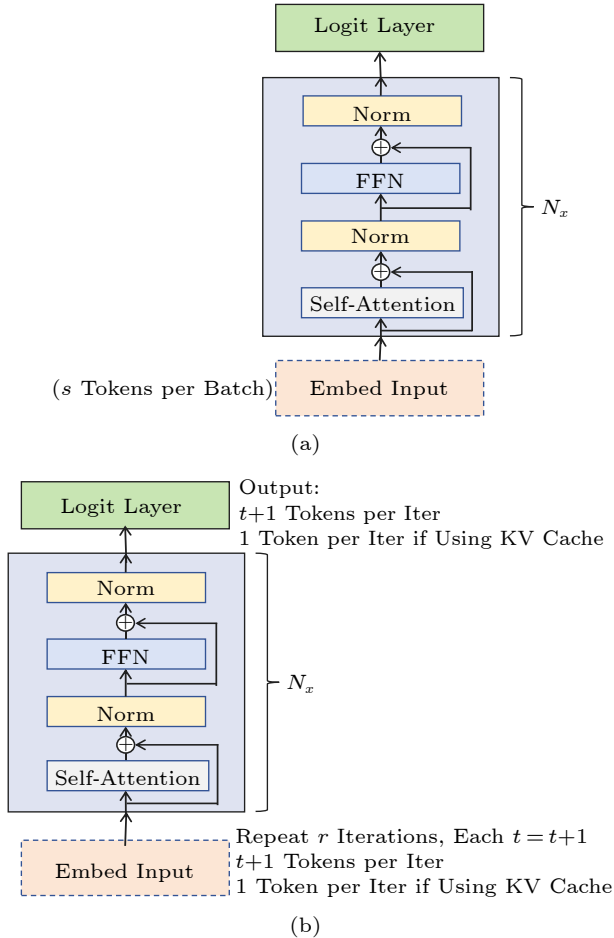


Fig.4. Comparison of training and inference computations for the auto-regressive decoder-only model. (a) Training. (b) Inference.

2.3 Model Analysis

2.3.1 Attention & FFN

Following the launch of the Transformer, although developments (such as MoE, layer normalization, positional encoding, and activation function) have significantly refined and optimized the Transformer architecture, the fundamental computational components of LLMs, which are built on attention and feed-forward networks (FFN), remain unchanged. At the same time, the values of hyperparameters do not affect the behavior of each basic operator, but only the specific values being computed in the operator.

Our subsequent analysis centers on the attention and FFN modules within the decoder block. As shown in Fig.5, each decoder block is comprised of a multi-head self-attention mechanism and a fully-connected feed-forward network, and the FFN comes after the attention mechanism. Both of them employ residual connections and normalization to enhance the network's convergence capability^[32].

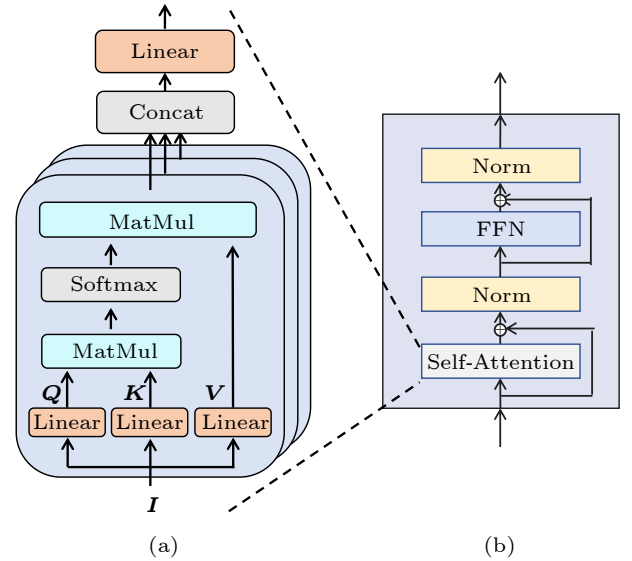


Fig.5. Self-attention in a decoder block. (a) Multi-head self-attention mechanism. (b) Decoder block.

The attention mechanism operates with three inputs: Query, Key, and Value. It calculates the similarity between different parts of the input sequence (often through the dot product of Query and Key), effectively weighting the importance of different parts, thereby focusing on essential information while disregarding the less important, significantly enhancing its ability to handle information within sequences. The computational process can be represented by mathematical formulas, where Q , K , and V are matrices

standing for Query, Key, and Value, respectively, as follows:

$$A_{\text{scale}}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^T}{\sqrt{d_k}}\right)\mathbf{V}.$$

The prevalent structure employed in decoder blocks is multi-head self-attention, as shown in Fig.5(a). Initially, the self-attention mechanism, or intra-attention, captures the correlations within the input sequence. The components $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ are generated from the same input source matrix ($\mathbf{I}$) through linear transformations. Then, attention is computed across multiple subspaces (i.e., multiple heads) independently, enabling simultaneous capture of various information facets of the input. Finally, the aggregated outputs from each head, combined via a linear layer, form the multi-head attention (MHA) output. This mechanism enables the model to capture complex relationships within sequences more effectively.

FFN processes each token's state information independently after aggregating the results of the MHA. It typically consists of two fully-connected (i.e., linear) layers, with an activation function like ReLU incorporated between two layers. Its computation can be represented as:

$$\text{FFN}(x) = \text{ReLU}(x\mathbf{W}_1 + b_1)\mathbf{W}_2 + b_2.$$

FFN executes relatively straightforward matrix operations within the Transformer architecture, which makes it well-suited for acceleration using the matrix function unit (MFU). The FFN is a major contributor to both the parameter count and operation load in LLM, due to the operator composition in the Transformer architecture and the typically large hidden di-

mension. This extensive-scale matrix computation exceeds the capacity of traditional on-chip SRAM. Achieving high-performance computational kernels, especially for MFU, is thus essential. Specifically, the weight shape of the first fully connected layer in FFN that is derived from the classical transformer is typically $(h, 4h)$, where h represents the hidden dimension. However, a different design is adopted in some models, such as LLaMA^[34] and Llama 2^[36], where the first fully connected layer uses a hidden dimension size of $2.7h$ instead of the typical $4h$.

2.3.2 Operational Intensity

In order to evaluate the bottlenecks of the Transformer more effectively, it is essential to break down its computation steps, and evaluate its number of arithmetic operations and computational intensity. The number of floating-point arithmetic operations indicates the model's demand for computational resources, while operational intensity (or arithmetic intensity) measures the level of computational intensity per memory access. Operations with a high operational intensity value are regarded as compute-intensive, while those with a low operational intensity value are considered memory-bound. Operational intensity is calculated as the ratio of the total number of arithmetic operations (AOPs) to the total number of memory access operations (MOPs), as follows:

$$\text{OperationalIntensity} = \frac{\#AOPs}{\#MOPs}.$$

Decomposition of Computation. As Table 2 shows, the computation of one decoder block can be divided into several steps, noting that we ignore certain oper-

Table 2. Details of Computations within a Decoder Block During Forward Propagation

Computation	Input 1	Input 2	Output	Number of Operations	Operational Intensity
1.1 $\mathbf{Q} = \mathbf{I}\mathbf{W}_Q$	(b, s, h)	(h, h)	(b, s, h)	$2bsh^2$	$1/(\frac{1}{h} + \frac{1}{2bs})$
1.2 $\mathbf{K} = \mathbf{I}\mathbf{W}_K$	(b, s, h)	(h, h)	(b, s, h)	$2bsh^2$	$1/(\frac{1}{h} + \frac{1}{2bs})$
1.3 $\mathbf{V} = \mathbf{I}\mathbf{W}_V$	(b, s, h)	(h, h)	(b, s, h)	$2bsh^2$	$1/(\frac{1}{h} + \frac{1}{2bs})$
2. $\mathbf{Q}, \mathbf{K}, \mathbf{V}$ preparation	Reshaping from (b, s, h) to (b, s, n, d) , and then transposing to (b, n, s, d)				
3. $\mathbf{W}_{\text{attn}} = \mathbf{Q}\mathbf{K}^T$	(b, n, s, d)	(b, n, s, d)	(b, n, s, s)	$2bs^2h$	$1/(\frac{1}{s} + \frac{1}{2d})$
4. $\mathbf{W}_{\text{attn}}\mathbf{V}$	(b, n, s, s)	(b, n, s, d)	(b, n, s, d)	$2bs^2h$	$1/(\frac{1}{s} + \frac{1}{2d})$
5. Concat	Transposing from (b, n, s, d) to (b, s, n, d) , and then reshaping to (b, s, h)				
6. Linear	(b, s, h)	(h, h)	(b, s, h)	$2bsh^2$	$1/(\frac{1}{h} + \frac{1}{2bs})$
7. FFN-part1	(b, s, h)	$(h, 4h)$	$(b, s, 4h)$	$8bsh^2$	$1/(\frac{5}{8h} + \frac{1}{2bs})$
8. FFN-part2	$(b, s, 4h)$	$(4h, h)$	(b, s, h)	$8bsh^2$	$1/(\frac{5}{8h} + \frac{1}{2bs})$

Note: Each row represents an operation, with Input 1, Input 2, and Output indicating the tensor shapes of the two inputs and their corresponding output. Here, b is the batch size, s is the sequence length, h is the size of hidden state, n is the number of attention heads, and d is the dimension of the key.

ations, such as residual connections and normalization, and assume that the off-chip memory space is large enough. Furthermore, we have disregarded arithmetic operations of some operators (Dropout and Softmax), because they generally do not run on the more powerful MFUs of accelerators. Instead, they are handled by the less powerful vector function units (VFUs) and have a constant level operational intensity. The steps are specified as follows.

Step 1. Linear transformations are performed on the input matrix I to obtain Query (Q), Key (K), and Value (V).

Step 2. Reshaping and transposing are performed in preparation for the MHA calculation.

Step 3. Batch matrix multiplication is carried out between Q and K for each head, followed by scaling and applying the softmax function to obtain the attention weight matrix W_{attn} (i.e., attn. weight).

Step 4. Batch matrix multiplication is carried out between W_{attn} and V for each head.

Step 5. The outputs of each head are concatenated and then processed for transposing and reshaping to match the original layout.

Step 6. A linear transformation is applied to integrate the outputs of the each head’s attention.

Step 7. The first linear transformation of the FFN is computed with increased hidden state size.

Step 8. The second linear transformation of the FFN is computed, and the output shape is returned to its original size.

Arithmetic Operations. It can be concluded that for a decoder block, the total number of operations during forward propagation is “ $24bsh^2 + 12bs^2h$ ” by adding up the parts in Table 2. During the process of backward propagation, double the computational workload is required due to the computation of gradients for activation value and model weight. As shown in Table 3, for a network with l decoder blocks, the floating-point operations required for one batch during training are “ $72bsh^2 + 12bs^2h$ ”.

Table 3. Number of Arithmetic Operations for a Model with l Decoder Blocks

Propagation	Operation
Forward	$24bsh^2 + 4bs^2h$
Backward	$48bsh^2 + 8bs^2h$
Total	$72bsh^2 + 12bs^2h$

Note: b means the batch size, s represents the sequence length, and h equals the size of the hidden state.

Operational Intensity Analysis. In the computational breakdown presented in Table 2, step 1 and

step 6 exhibit identical operational intensity, with the function $1/((1/h) + (1/2bs))$ is labeled as “dense 1”. The final two steps are the two parts of the FFN computation and also have the same operational intensity, expressed as $1/((5/8h) + (1/2bs))$, referred to as “dense 2”. Additionally, step 3 and step 4 share the same operational intensity, given by $1/((1/s) + (1/2d))$, termed “attention”. Using the formulas derived from the analysis in Table 2, we further explore the effects by controlling specific variables.

As shown in Fig.6(a), the operational intensity of attention is not affected by batchsize and consistently lower than that of A100 and H100 chips, suggesting it as memory-bound. In contrast, operations like “dense 1” and “dense 2”, with their higher operational intensity, fall into the compute-bound category. Therefore, it can be inferred that attention may have lower execution efficiency compared with other compute-bound operations. Furthermore, as shown in Fig.6(b), although attention accounts for a relatively small proportion of total operations, its percentage significantly increases with the growth of sequence length, reaching over 20%, implying rising time consumption in transformer computations.

Considering its lower operational intensity, attention is likely to become a performance bottleneck. In

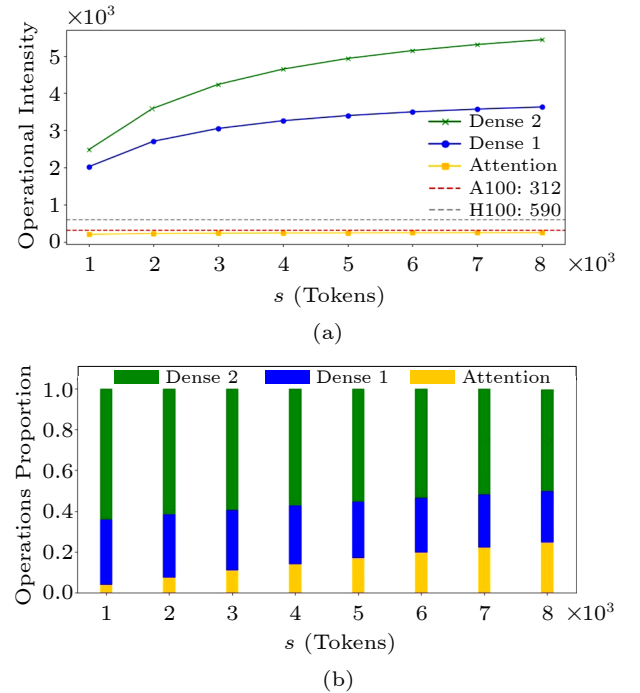


Fig.6. (a) Comparison of operational intensities at different sequence lengths. (b) Proportion of arithmetic operations at different sequence lengths. The sequence length is represented by s and is measured in tokens, while the other parameters are held constant: the batch size is 2, the size of the hidden state is 4096, and the dimension of the key is 128.

fact, attention is responsible for over 50% of the transformer latency^[65, 66], and internal operations like Dropout and Softmax, which are ignored when evaluating operational intensity, actually contribute to more than 50% of the attention latency^[65–68]. These insights highlight the imperative to prioritize the optimization of attention mechanisms within both software and hardware designs.

Auto-Regressive Situation. By applying the decomposition process outlined in Table 2, the operational intensity of each part during auto-regressive inference can be determined, as shown in Table 4. The operational intensity in this table are inferred from the forward propagation during inference. When the KV cache is performed, the shape of the input x is $(b, 1, h)$. Notably, these intensities significantly decrease compared with those encountered in long-sequence training, especially with small batches, where the memory access bottleneck becomes more apparent.

Table 4. Operational Intensity of Auto-Regressive Inference with KV Cache

Computation	Operational Intensity
$q = xW_Q$	$1/(\frac{1}{h} + \frac{1}{2b})$
$k = xW_K$	$1/(\frac{1}{h} + \frac{1}{2b})$
$v = xW_V$	$1/(\frac{1}{h} + \frac{1}{2b})$
$W_{\text{attn}} = qK^T$	$1/(\frac{1}{2} + \frac{1}{2s} + \frac{1}{2d})$
$W_{\text{attn}}V$	$1/(\frac{1}{2} + \frac{1}{2s} + \frac{1}{2d})$
Linear	$1/(\frac{1}{h} + \frac{1}{2b})$
FFN-part1	$1/(\frac{5}{8h} + \frac{1}{2b})$
FFN-part2	$1/(\frac{5}{8h} + \frac{1}{2b})$

Note: b means the batch size, s represents the sequence length, h equals the size of the hidden state, and d indicates the dimension of the key.

3 LLM Hardware Platforms

This section presents an overview of the trends and challenges in AI computing systems for LLMs from a hardware infrastructure perspective. It starts with single computing node configured with multiple accelerators, detailing the internal topology structure of heterogeneous processors. Then, computing clusters are constructed by incorporating multiple computing nodes, where transport and topology are important factors affecting the ability to scale out. We highlight the growing gap between LLM complexity and accelerator capabilities, emphasizing the need for increased accelerator interconnectivity.

3.1 Scale-up Computing Node

A typical LLM computing node is a heterogeneous computing server, primarily comprising host-side components like CPUs and memory, and device-side accelerators. The most straightforward way to boost node performance is to employ more powerful accelerators and increase their number. However, the internal topology and affinity of computing nodes critically influence communication and memory access, significantly impacting data throughput and access latency. While traditional deep learning models may be less sensitive to these factors, LLMs are particularly affected. For instance, although the NVIDIA A100 PCIe GPU and A100 SXM4 GPU exhibit similar peak computational power, NVIDIA’s MLPerf v2.1 results^⑨ illustrate that in BERT model training, the A100 SXM4, equipped with high-speed interconnection, substantially outshines the A100 PCIe, cutting down network training convergence time from 31 to 23 minutes.

3.1.1 Accelerator

In recent years, the performance of accelerators in heterogeneous computing has been continuously improved. By examining key parameters, we can observe several ongoing trends.

1) *Matrix Function Unit.* The Cambricon^[69] architecture laid the foundation in 2015 by suggesting that deep learning processors should incorporate matrix function units (MFUs), vector function units (VFUs), and scalar function units (SFUs) simultaneously. This design has been embraced by subsequent research and accelerators. Accelerators designed for deep learning and LLMs typically feature matrix function units, embracing capabilities for multiple data precisions. These systems have evolved significantly, starting from supporting FP16 tensor operations to now facilitating a range of tensor computations with specialized data types like BF16, TF32, and FP8, which are customized for deep learning applications.

2) *Transistor Count.* Several years back, the transistor count in typical GPUs (such as NVIDIA’s K40, M40, P100) was close to 10 billion. Benefiting from advanced semiconductor fabrication processes and chiplet design technologies, AI chips in the LLM era have significantly progressed. A prime example is the

^⑨MLPerf: <https://mlcommons.org/benchmarks/training/>, Jan. 2025

AMD MI300 GPU released in 2023, which integrates multiple chiplets and incorporates over 150 billion transistors.

3) *Cache Size*. As the number of processing elements in accelerators grows, there is a corresponding increase in on-chip memory capacity. For instance, while the NVIDIA V100 accelerator, released in 2017, features a 6MB L2 cache, this capacity saw a significant jump to 50 MB in the NVIDIA H100 accelerator released in 2022. Similarly, other accelerators have been equipped with large-capacity cache or scratch-pad memory (SPM) to enhance their performance and efficiency.

4) *Memory*. In the training of LLMs, accelerators must prioritize off-chip memory bandwidth and capacity. Consequently, accelerators in the era of LLM widely utilize high bandwidth memory (HBM), a 3D-stacked synchronous dynamic random-access memory. HBM has evolved from the first generation to HBM3e, continuously increasing memory bandwidth and capacity. These developments enable accelerators to maintain high memory bandwidth while also offering increased off-chip memory capacity. For instance, the MI300 released in 2023 is equipped with 192 GB of off-chip memory and has more than 5 TB/s of bandwidth.

5) *Interconnection*. The bandwidth provided by accelerators using PCIe protocol interconnects is insufficient for training LLMs. Current accelerators use independent links and private protocols for intercon-

nection. A notable example is NVIDIA’s NVLink protocol, which achieves a substantial leap to 900 GB/s of bidirectional bandwidth. Accelerator interconnection development focuses on expanding bandwidth, with further discussion on topology in [Subsection 3.1.2](#).

As shown in [Fig.7](#), in recent years, the parameter size of LLMs has seen an exponential increase, scaling up by several orders of magnitude. This rapid growth in model complexity starkly contrasts with the relatively slower pace of advancements in mainstream accelerators, particularly in terms of transistor count and memory capacity. As a result, the “gap” between the development of LLMs and memory technology has been widening, posing significant challenges to accommodating these expansive models on current accelerators. Moreover, the computational operations for training these large-scale models correlate directly not only with the number of model parameters but also with the size of training datasets. This escalating requirement for computational resources is increasingly outpacing the supply of available resources in hardware, creating a widening chasm between demand and provision.

To bridge this gap, a systematic approach involving both hardware and software optimizations is imperative. Techniques such as distributed computing optimizations, enhanced scheduling, memory optimization, quantization, and model compression become crucial. These methodologies aim to construct a more efficient computational system for LLMs, ad-

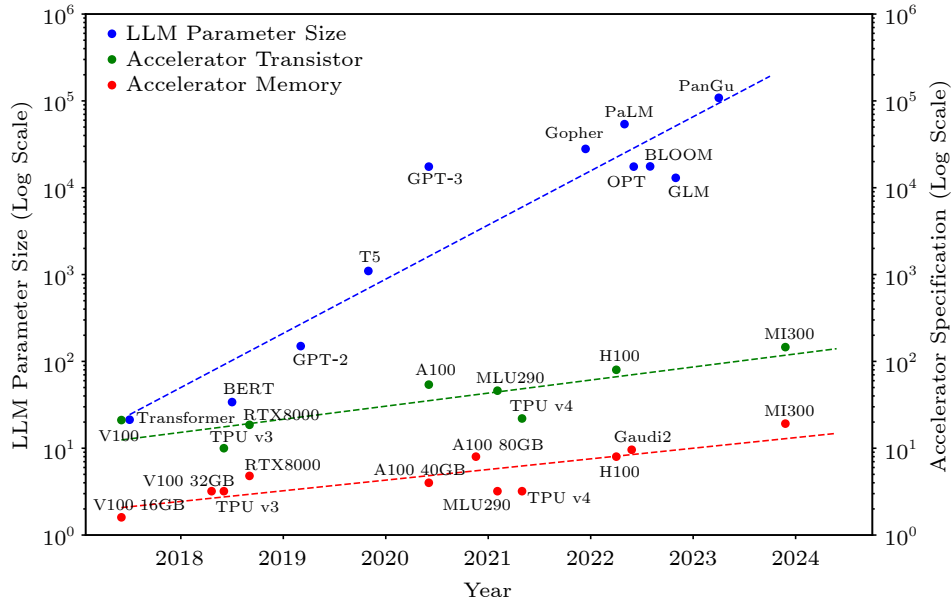


Fig.7. Growing gap between LLMs and accelerators (including transistor count and memory capacity). V100, RTX8000, A100, and H100 are released by NVIDIA. TPU v3 and TPU v4 belong to Google. MI300, Gaudi2, and MLU290 are products by AMD, Intel, and Cambricon Technology, respectively.

addressing the constraints imposed by current hardware limitations and leveraging the potential of software innovations to manage and mitigate the complexities involved in training and deploying LLMs.

3.1.2 Affinity & Interconnection Topology

One major challenge in building computing nodes is interconnection between CPUs and accelerators, due to the requirement for cooperation between processors of diverse architectures.

In the process of interconnecting devices, a key consideration is the affinity among them, meaning that computational tasks should be kept close to certain specific hardware resources, such as processors, memory, accelerators, and I/O devices. For example, positioning accelerators and network interface cards (NICs) under the same PCIe switch allows for GPU P2P direct intercommunication and the use of GPU direct RDMA technology, bypassing CPU forwarding and enhancing system efficiency. Minimizing unnecessary cross-CPU or cross-memory accesses is crucial, as these can potentially impact the overall throughput of the application. Proper configuration of device affinity can improve system performance and response time. In deep learning workloads, intense data interaction between the host-side CPU and device-side accelerator is common, and the interconnection topology between the host and device can be co-designed to suit workload requirements, leveraging the flexibility offered by PCIe technology.

Specifically, the interconnection topologies include single-root topology (i.e., common or cascade), dual-root topology (i.e., balance), and direct-attached, as shown in Fig.8. Training nodes often use a common topology, while those set up with high-speed ac-

celerator interconnects utilize a balanced topology, and inference-dedicated nodes employ a direct-attached topology. Different affinities and topologies predominantly influence three aspects: 1) the overall communication bandwidth between CPUs and accelerators, 2) the bandwidth available for inter-accelerator communication, and 3) the latency in communication between accelerators[70, 71]. These factors directly impact the performance across various application scenarios, making the choice of an appropriate topology essential for enhancing computational efficiency and optimizing resource allocation.

Another major challenge in building computing nodes is enhancing the interconnectivity of accelerators, which is crucial for high-speed communication in LLM computations.

Data-center accelerators universally support high-speed interconnect technologies, such as NVIDIA's NVLink and Cambricon's MLU-Link. Tailored to more specific application scenarios, these technologies overcome the PCIe protocol's limitations, and offer over 10 times the transfer bandwidth of PCIe 4.0 $\times$ 16. The prevalent interconnect topologies include structures such as rings, 2D torus, and hypercubes. As shown in Fig.9, to balance cost and benefit, interconnect networks may vary from local links among a few accelerators to system-wide links. For instance, a server with eight accelerators could have diverse network configurations, such as four small rings for pairwise connections (i.e., Fig.9(a), exemplified by NVIDIA A100 PCIe accelerators linked via NVLink Bridge), two medium rings each connecting four accelerators (i.e., Fig.9(b), similar to the setup of Cambricon MLU370 PCIe accelerators connected through MLU-Link Bridge), or a single large ring configuration, as

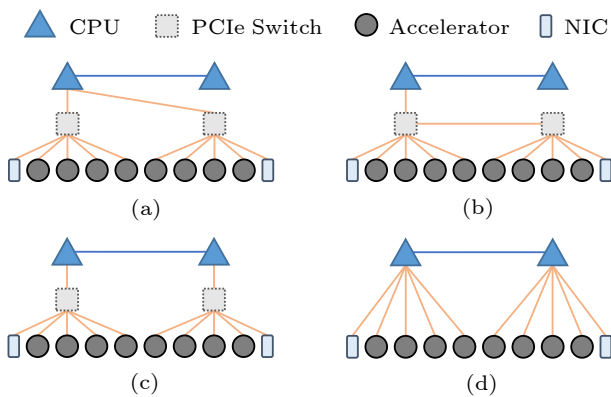


Fig.8. Interconnection topologies for a server with two CPUs and eight accelerators. (a) Common. (b) Cascade. (c) Balance. (d) Direct-attached.

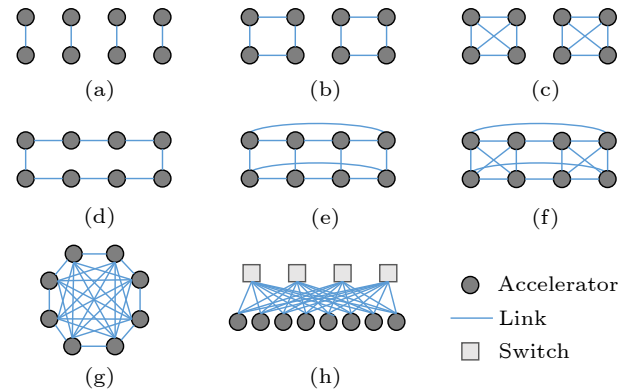


Fig.9. Overview of accelerator interconnect topologies within a server with eight accelerators. (a) Four rings. (b) Two rings. (c) Two fully-connected. (d) Ring. (e) Torus. (f) Hypercube. (g) Fully-connected. (h) Switch network.

well as 2D torus, hypercube. Switch networks enable full-speed arbitrary switching, providing maximal connection rates between accelerators and good scalability. Yet, not all accelerator manufacturers are able to implement this topology, often due to technological and cost limitations. Therefore, other alternatives must be considered. The optimization of communication libraries is very critical at this time, and the underlying implementation of algorithms like all-reduce must be decomposed into multiple different path loops.

Currently, the interconnection between processors and accelerators is not limited to the traditional PCIe protocol. For instance, in the Summit supercomputer, IBM's Power CPUs are interconnected with GPUs via NVLink^[72], facilitating more rapid data exchanges. NVIDIA's Grace Hopper superchip utilizes NVLink Chip-to-Chip interconnect technology, directly connecting CPU and GPU chips, thereby enhancing data processing speed and capabilities. Additionally, the CXL (Compute Express Link) technology is poised to supersede PCIe for processor-accelerator interconnections^[73]. Moreover, even though most servers are manufactured with a fixed topology, there are servers that enable dynamic topology adjustments via the control of PCIe switches. This flexibility in the topology allows these servers to more efficiently cater to varying computational requirements.

3.2 Distributed Scale-out Cluster

Scale-out clusters, which expand by adding scale-up computing nodes, address computational demands by distributing tasks across multiple high-speed connected servers and devices, as shown in Fig.10. A unified data storage system is required in the cluster to facilitate data access among nodes. The deployment

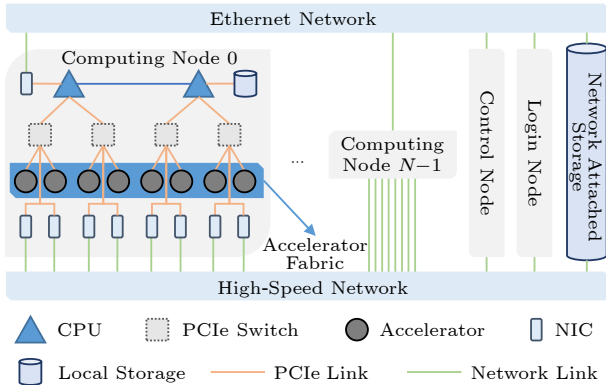


Fig.10. Scale-out cluster with numerous AI computing scale-up nodes.

of multiple sets of interconnected communication networks ensures high-speed connectivity between nodes. While not all algorithms suit this cluster design, LLM computations thrive under it. Effective network transport and network topology are vital for distributed scale-out clusters due to the increased sensitivity of LLM networks to communication capabilities and their substantial resource requirements compared with general AI computations. Previously, training computer vision models used a few hundred GPUs and normal high-speed networks; now, LLMs training employs up to ten thousand GPUs and ultra high-speed networks, significantly raising system scaling requirements.

3.2.1 Network Transport

One challenge in building computing clusters is achieving high-bandwidth network transport, which is crucial for the extensive data communication inherent in distributed training. As the LLM parameter size grows, so does the demand for communication bandwidth. Generally, more NICs are configured for computing nodes, along with increased bandwidth for each card. In training CV models such as ResNet 50, Inception V3, and VGG16 on 512 GPUs, the throughput difference between 25 Gb RoCE and 100 Gb RDMA networks is minimal, indicating that a single NIC is adequate for traditional HPC servers and pre-LLMs era AI servers. However, for advanced AI computing clusters designed for LLMs, each accelerator card may be equipped with a dedicated NIC, significantly enhancing communication bandwidth and throughput. In the evolution from NVIDIA DGX-1 to DGX-A100 and DGX-H100, each configuration includes two CPUs and eight accelerators, with the number of NICs at 4, 8, and 8 respectively, and the bandwidth of each card has also been increased progressively from 100 Gbps to 200 Gbps and 400 Gbps.

Another challenge in building computing clusters is reducing the overhead of network transport, as traditional kernel-based TCP/IP performs poorly in satisfying LLMs training. Therefore, it is necessary to reduce kernel overhead and implement efficient, accelerator-centric communication systems. Typically, network transport in traditional data-centers rely on Ethernet and TCP/IP, and Linux-based operating systems divide memory into the user space and kernel space. This architecture necessitates multiple memory copies and extensive network protocol processing for both data sending and receiving, consum-

ing substantial CPU resources. Multilayer protocol processing consumes a significant amount of CPU resources, especially in high-speed network communications. This consumption limits CPU availability for computational tasks. Additionally, context switching between the user and kernel modes can negatively impact application performance.

To address this issue, remote direct memory access (RDMA) is extensively employed, and InfiniBand and RDMA over Converged Ethernet (RoCE) represent the most commonly used RDMA techniques. RDMA enables direct data transfers between the RAMs of different computers, offering advantages over TCP/IP such as zero-copy and kernel bypass. Zero-copy reduces data copying between the user and kernel spaces, and kernel bypass reduces software overheads, all without kernel involvement from either system. As a result, RDMA has low latency and low CPU overhead characteristics, as shown in Fig.11. Additionally, GPU direct RDMA (GDR) or similar technology is also required to avoid copying data from accelerator memory to CPU memory before RDMA communication. It enables direct data transfer from GPU to NIC, bypassing CPU memory and kernel overhead, which is pivotal for heterogeneous systems.

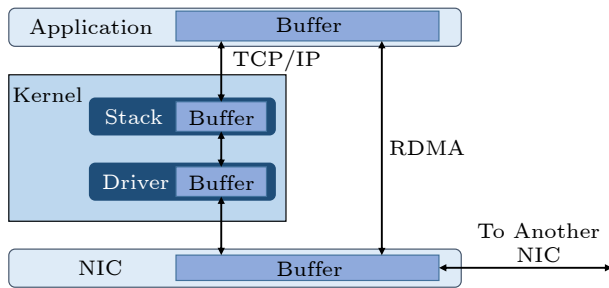


Fig.11. TCP/IP v.s. RDMA.

Related research^[74] shows that for training models such as GPT-2, BERT-Large, and T5 across eight nodes, RDMA-100 Gbps significantly outperforms both IPoIB-100 Gbps (TCP/IP over InfiniBand NIC) and TCP/IP-10 Gbps, reducing training iteration times by factors of 2.51 and 4.79, respectively. Therefore, RDMA-related techniques for accelerators are very important for LLMs training.

3.2.2 Network Topology

Network topology is also crucial for scale-out computing clusters, directly affecting communication efficiency and scalability. As shown in Fig.12, prevalent

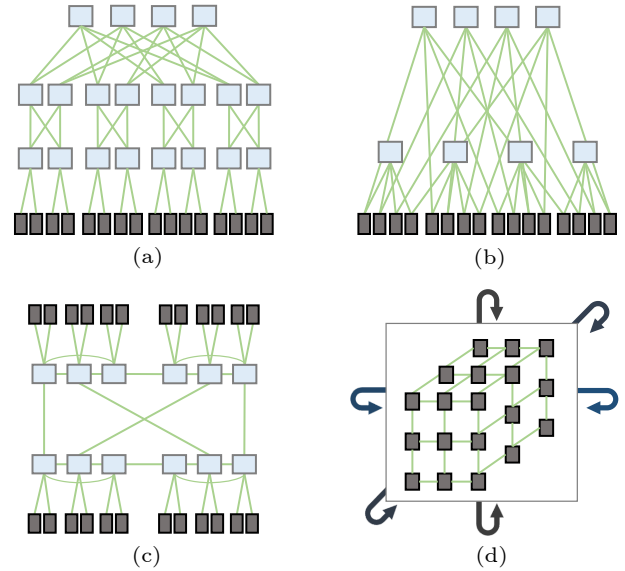


Fig.12. Comparison of different network topologies. (a) Fat-tree. (b) BCube. (c) Dragonfly. (d) 3D torus. Note that each access point here usually refers to an NIC or an accelerator with external interconnect interface.

network topologies include fat-tree or (leaf-spine), BCube, dragonFly, 3D torus, with technical focus typically on features such as network diameter, routing algorithms, flow control, and average latency. For instance, NVIDIA DGX SuperPod utilizes a hierarchical fat-tree structure, whereas Google TPU^[12, 75] employs a flattened 3D torus structure.

According to an industry viewpoint, InfiniBand might represent approximately 20% of the total cost of a computing cluster, and the more access points, the higher the percentage. Balancing cost and performance in network design is challenging. Non-blocking topologies offer superior performance but are more costly with more network access points, while cost-effective topologies may sacrifice some performance. As shown in Fig.13, accelerators in the same server create a non-blocking ultra-high bandwidth domain via accelerator fabric interconnection. Some servers form the same sub-clusters enabling non-blocking high bandwidth connections. However, the global network topology is intentionally over-subscribed to manage costs, leading to an inability to maintain line-rate communication across all servers. Such localized bandwidth bottleneck places higher demands on hardware and software infrastructure such as advanced job partition, topology-aware communication libraries, and hardware congestion control.

Overall, an effective network topology is crucial for maintaining efficient communication and handling extensive LLM workloads. As the demand for

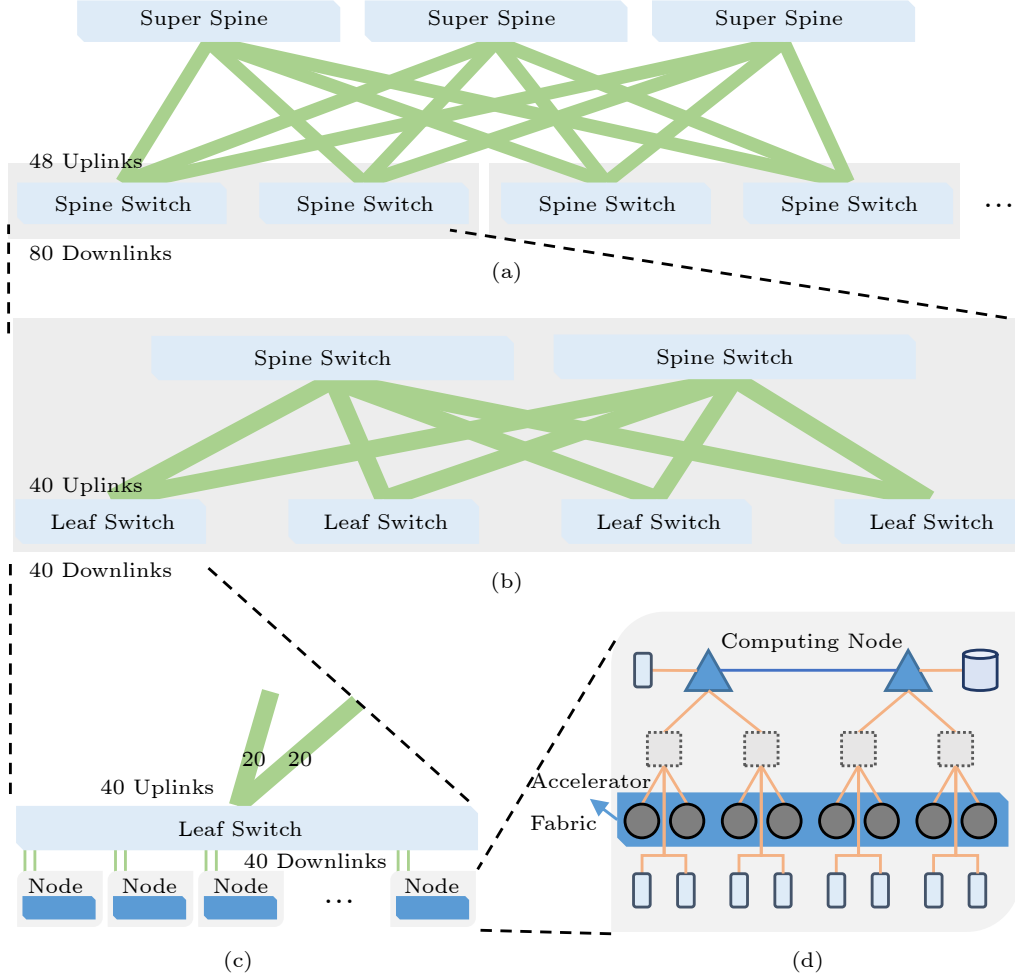


Fig.13. Hierarchical but partially blocking network topology. (a) Super spine network with 3:5 oversubscription. (b) Spine-to-leaf network with 1:1 oversubscription. (c) Twenty nodes in a single sub-cluster. (d) Single node within ultra-high bandwidth domain. Note that each node requires connections to four leaf switches, with two interconnections per leaf switch.

LLMs computing and data processing grows, scaling a network with additional computing nodes becomes essential, requiring robust network topology designs. Networks comprising ten thousands of NICs face special challenges^[76] such as minimizing forwarding delays by controlling topology diameter, reducing hash conflicts in equal-cost multi-path routing, managing congestion effectively, and balancing communication performance with cost-effectiveness. In addition, the implementation of a communication collective library must consider the affinity of the NICs and accelerators^[70], the network transport mechanisms (e.g., GDR/RDMA vs. TCP), and the network topology^[77] in order to optimize performance.

4 LLM Software and Scheduling

This section delves into executing and scheduling LLM algorithms on hardware infrastructures from a software perspective. Initially, we collect and briefly

introduce the current LLM programming frameworks and highlight their characteristics. Then, we discuss software optimization techniques, which can be divided into three major categories: 1) how to run LLM distributively (including data, tensor, and pipeline parallelism), and achieve efficient job partition and enhance fault tolerance; 2) how to overcome the challenges inherent in memory management, which entails deploying recomputation and offloading to alleviate peak memory usage; 3) how to enhance the computational efficiency, which includes using quantization, leveraging sparsity, and implementing efficient kernel. Finally, we briefly introduce the similarities and differences between inference and training.

4.1 Programming Frameworks

With the continuous expansion in the size of models and data volumes within the LLMs domain, tradi-

tional deep learning programming frameworks (such as outdated static graph frameworks and the original versions of PyTorch) no longer meet the specific requirements of these models executions. To overcome the distinctive challenges such as tensor parallelization, memory management, and communication optimization, some high-level programming frameworks for LLMs have been introduced. Notably, these include Microsoft's DeepSpeed, NVIDIA's Megatron-LM, and Meta's FSDP, along with other derivative frameworks. These advanced frameworks primarily leverage PyTorch-based programming environments and are executed on hardware platforms supported by software stacks within computing systems.

Deep learning programming frameworks are mostly based on the computational graph mechanism. Now, eager mode (i.e., Define-by-Run^[78]) frameworks, such as PyTorch and JAX, have become mainstream. Conversely, static graph mode (i.e., Define-and-Run^[78]) frameworks, including Caffe, TensorFlow 1.x, MXNet, and CNTK, are knocked out in the era of LLMs. These outdated frameworks rely heavily on data parallelism in distributed training when addressing the requirements of large-scale parallel computing. Yet, such a single parallelism strategy often fell short in efficiently processing LLMs, particularly under the constraints of accelerator memory capacities.

To facilitate advanced parallelism like tensor parallelism and pipeline parallelism, developments such as Mesh-TensorFlow^[79] and Torchpipe^[80] emerged. Despite offering innovative parallel mechanisms, their flexibility is still limited for complicated parallel strategies. In tackling the issue of excessive memory space taken up by redundant model states in data parallelism, ZeRO^[81] and FSDP implemented a distributed approach to storing optimizer states, rather than maintaining multiple copies. This strategy significantly reduces the overall memory space requirement.

Programming frameworks for LLMs should support appropriately complex parallel strategies while ensuring efficient execution of models. With the introduction of projects like Megatron-LM^[17, 18], programming frameworks that support hybrid parallelism (i.e., a combination of multiple parallel methods) appeared, including DeepSpeed^[13] and Megatron-LM, and subse-

quently led to the development of extensions such as Megatron-DeepSpeed^⑩, Megatron-LLaMA^⑪, and veGiantModel^⑫. Concurrently, domestic frameworks such as PaddleFleetX, OneFlow^[82], and easy parallel library (EPL)^[83] have also adopted hybrid parallelism. This advancement has significantly eased the programming complexities associated with LLMs and enabled the successful training of a variety of LLMs.

As complex parallel strategies have been realized, the development of programming frameworks currently focuses on lowering the training costs of LLMs, easing the complexity of predefined strategies, and increasing execution efficiency, which encompasses new areas like offloading, elastic training, and automatic hybrid parallelism^[84-89]. The main features and supported parallel mode of these frameworks are listed in Table 5, and except for the first two older frameworks, the remaining frameworks, numbering over a dozen, are able to run the LLMs. It is noteworthy that all the programming frameworks mentioned above support training, and their source codes are accessible on the Internet^⑬.

4.2 Enhancing Distributed Computing: From Parallel Optimization to Fault Tolerance

As distributed computing of LLMs training scales to unprecedented levels, involving setups like 10 000 accelerators, new challenges emerge that are not present in smaller setups. These challenges include the necessity for automated tools to manage complex parallel strategies and enhanced fault tolerance measures to handle the increased system complexity. Consequently, the focus for distributed computing in LLMs will shift towards optimizing parallel strategies, job partitioning, and fault tolerance mechanisms.

Initially, we analyze the communication and operational intensity along with a brief description of the parallel strategies. Then, we list work related to optimizing parallel strategies, job partition, and auto parallel, as well as fault tolerance mechanisms. These advancements not only enhance computational efficiency and simplify strategy configuration, but also significantly improve the fault tolerance capabilities, which are critical for the successful training of LLMs.

⑩<https://github.com/microsoft/Megatron-DeepSpeed>, Jan. 2025.

⑪<https://github.com/alibaba/Megatron-LLaMA>, Jan. 2025.

⑫<https://github.com/volcengine/veGiantModel>, Jan. 2025.

⑬Links provided in the supplement "4178_Supplement_document.pdf" at <https://jcst.ict.ac.cn/article/doi/10.1007/s11390-024-4178-1>, Jan. 2025

Table 5. LLMs Training Programming Frameworks

Programming Framework	Basic Source Component	Feature	Parallel Mode
Mesh-TensorFlow ^[79]	TensorFlow	Distributed tensor computation	MP
Torchgpipe ^[80]	PyTorch	Micro-batch pipeline by Gpipe	PP
Fairscale (Meta) ^[40,43]	PyTorch	Fully sharded data parallel (FSDP)	DP
Deepspeed ^[13-16] (Microsoft)	PyTorch	Zero redundancy optimizer (ZeRO ^[81]), ZeRO series	DP, MP, PP
Megatron-LM ^[17, 18] (NVIDIA)	PyTorch	Interleaved 1F1B pipeline, partitioned transformer	DP, MP, PP
Megatron-DeepSpeed Forked from Megatron-LM (Microsoft)	Forked from Megatron-LM	DeepSpeed integrated in Megatron	DP, MP, PP
Megatron-LLaMA (Alibaba)	Forked from Megatron-LM	LLaMA, improved sharding method, distributed checkpoint	DP, MP, PP
veGiantModel (ByteDance)	Megatron-LM and DeepSpeed	Improved communication, customized pipeline partitioning	DP, MP, PP
PaddleFleetX (Baidu)	PaddlePaddle	Hybrid parallel (4D), unified trainer	DP, MP, PP
OneFlow ^[82] (OneFlow)	OneFlow	Split, broadcast and partial-value (SBP)	DP, MP, PP
EPL or Whale ^[83] (Alibaba)	TensorFlow	Easy parallel annotation	DP, MP, PP
AxonN ^[84] (Axonn-AI)	PyTorch	Asynchrony and message-driven execution, CPU memory offloading	DP, PP
Varuna ^[85] (Microsoft)	PyTorch	Elastic training, low-bandwidth network, job morphing	DP, PP
Colossal-AI ^[86] (HPC-AI)	PyTorch	Multi parallelism strategies, memory offloading (PatrickStar)	DP, MP, PP
Merak ^[87]	PyTorch	Automatic model partitioner	DP, MP, PP
Bamboo ^[88]	DeepSpeed	Elastic training, redundant computation, preemptible instance	DP, PP
Oobleck ^[89] (UMich)	PyTorch, DeepSpeed, Merak	Elastic training, pipeline template, dynamic reconfiguration	DP, MP, PP

4.2.1 Distributed Computing Basis & Analysis

The distributed parallel strategies for deploying LLMs primarily include data parallelism, tensor parallelism, pipeline parallelism, and a combination of them, as shown in Fig.14 and discussed in the following text.

Data parallelism replicates identical copies of the entire neural network model, including all layers and parameters, across multiple devices. Each copy processes different data segments, ensuring balanced workloads and consistent processing time during training. After forward and backward propagation, devices exchange gradients synchronously or asynchronously for model updates. Despite facilitating high throughput, it strains huge memory resources due to redundant copies, especially with LLMs. Therefore, optimizations or alternative parallel computing strategies are necessary.

Tensor parallelism, or intra-operator parallelism, divides large-scale operators horizontally for distribu-

tion across multiple devices, processing the same data batch on all accelerators to minimize memory overhead from model replication. It partitions the operators according to balanced workloads, and addresses the memory constraints of a single accelerator. However, it creates inter-accelerator communication challenges, necessitating optimizations in operator segmentation and communication reduction to boost computational efficiency.

Pipeline parallelism, or inter-operator parallelism, distributes operators across devices, creating a pipeline by layer division. This method improves memory efficiency by avoiding duplicated activations, enabling larger model training by adding flexibly divided pipeline stages. However, it introduces “model parallel bubbles” due to dependencies and update strategies, causing device idle time. Inconsistent memory usage at different stages, and computational workload imbalance across stages may lead to inefficient utilization of computing resources.

^[4]<https://github.com/facebookresearch/fairscale>, Jan. 2025.

^[5]<https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/>, Jan. 2025.

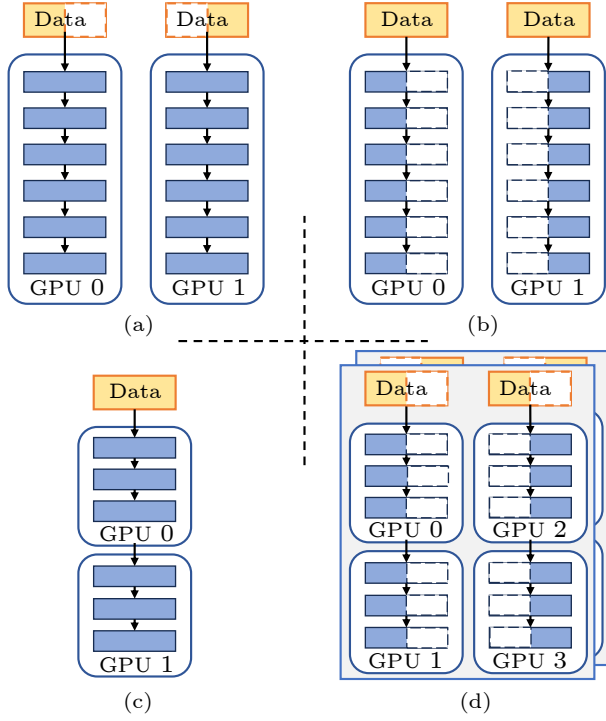


Fig.14. Distributed parallel strategies. (a) Data parallelism, DP. (b) Tensor parallelism, TP. (c) Pipeline parallelism, PP. (d) Hybrid parallelism.

Hybrid parallelism combines data, tensor, and pipeline parallelism, potentially with expert and sequence parallelism. Typically, tensor parallelism and pipeline parallelism operate orthogonally, which allows them to function concurrently without mutual interference, and their combination allows for running a complete LLM. Furthermore, data parallelism is also orthogonal to the above two parallelism approaches, suggesting that these three parallel strategies can be effectively combined. Hybrid parallelism can be denoted by (d, t, p) , with d for data paral-

lelism replicating the model into d copies, t for tensor parallelism dividing operators across t accelerators, and p for pipeline parallelism segmenting the model into p stages. In addition to the above strategies, expert parallelism^[43, 90] and sequence parallelism^[91] can also be applied in orthogonal. Expert parallelism (EP) enables MoE LLMs such as DeepSeek V3 to distribute the computation of expert sub-networks to multiple devices in parallel. Sequence parallelism (SP) divides long input sequences into smaller segments, processing them in parallel.

The introduction of parallel computing strategies aims to decompose the LLM operations on accelerators, address memory limitations, and accelerate computations by exploiting distributed computing. However, these strategies simultaneously incur additional communication overhead. Each parallel strategy leads to varying communication characteristics, as follows.

Data parallelism involves gradient data exchange among replicas that share parameters during gradient updates, primarily through ALL-Reduce operations. This process works by averaging and synchronizing the calculated gradients across replicas, which in turn roughly doubles the communication cost compared with the raw model weight data.

Tensor parallelism requires the transfer of activation values during each layer's propagation, using ALL-Reduce and sometimes ALL-Gather. As shown in Fig.15, it requires synchronization of duplicated activation values (i.e., Z_1 and Z_2) in forward pass, and synchronization of activation gradients in backward pass.

Pipeline parallelism focuses on transmitting activation values (forward) and activation gradients (backward) at network segmentation points, employ-

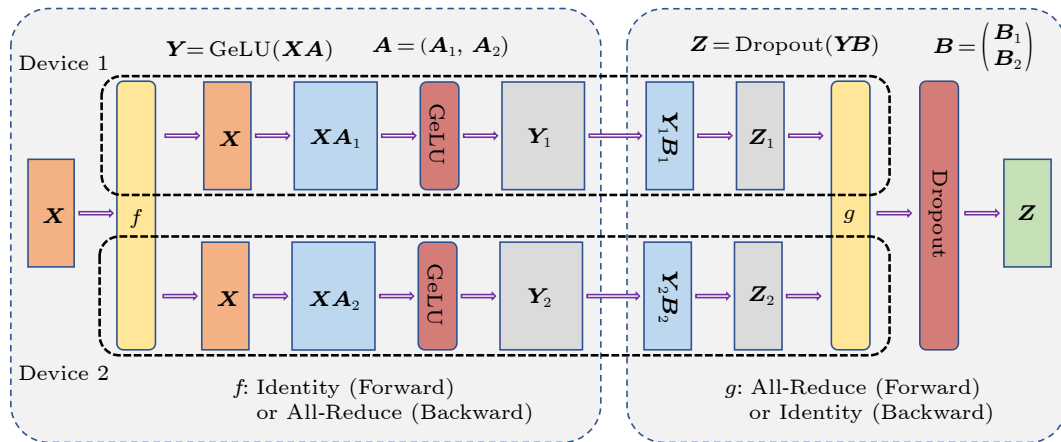


Fig.15. Tensor parallelism in MLP (Multi-Layer Perceptron), where f and g are defined as conjugate operations^[17, 18]. Note that the tensor shape of X and Z are both (b, s, h) .

ing point-to-point (P2P) communication to manage data flow between layers.

As shown in Table 6, the communication volume generated in each parallelism type is significant. Tensor parallelism generally leads to the highest communication cost and is better utilized within a group of ultra high-speed interconnect accelerators. The communication in LLMs training, besides large data transmission, also has the following characteristics: frequent communication across different parallelism types, and uneven communication distribution due to dependencies in propagation operators, causing temporal disparities in communication.

Table 6. Communication Generated When Executing the Training of a Batch with a Hybrid Parallel Strategy

Strategy	Content	Communication
DP	ParamGrads	$2Ndw$
TP	Act, ActGrads	$8bshtldmw$
PP	Act, ActGrads	$2bsh(p-1)dmw$

Note: ParamGrads means parameter gradients, Act means activation, and ActGrads means activation gradients. N represents parameter size, d represents the degree of DP, t represents the degree of TP, p represents the degree of PP, w represents data bit width, b represents batch size, s represents sequence length, h represents the size of the hidden state, and $m = B/(d \times b)$ represents the number of micro-batches.

Furthermore, tensor parallelism impacts operational intensity. Although model parallelism allows for the distribution of large operators across multiple GPUs, reducing the computational workload assigned to each accelerator, it can lead to a decrease in the operational intensity of several dense layers, potentially leading to reduced hardware utilization. Detailed information on operational intensity is provided in Table 7.

Table 7. Tensor Parallel Forward in Training

Computation	Operation per Accelerator	Operational Intensity
Q, K, V	$2bsh^2/t$	$1/(\frac{t+1}{2h} + \frac{1}{2bs})$
$W = QK^T$	$2bs^2h/t$	$1/(\frac{1}{s} + \frac{1}{2d})$
WV	$2bs^2h/t$	$1/(\frac{1}{s} + \frac{1}{2d})$
Linear	$2bsh^2/t$	$1/(\frac{t+1}{2h} + \frac{1}{2bs})$
FFN-part1	$8bsh^2/t$	$1/(\frac{t+4}{8h} + \frac{1}{2bs})$
FFN-part2	$8bsh^2/t$	$1/(\frac{t+4}{8h} + \frac{1}{2bs})$

Note: b means batch size, s means sequence length, h means the size of the hidden state, d stands for the dimension of the key, and t stands for the degree of TP.

4.2.2 Parallel Optimization

Building upon the parallel strategies of data, ten-

sor, and pipeline parallelism introduced previously, parallel optimization enhances the efficiency of deep learning model training by refining basic parallel strategies, as shown in Table 8.

Data parallelism faces significant challenges such as extensive cross-node communication, synchronizing gradient updates, and the inefficiency of redundant parameter memory. Enhanced communication methods tailored to cluster topologies, such as hierarchical ALL-Reduce^[92, 124, 125], or algorithm and system co-design^[126], can significantly improve communication ability. To mitigate gradient update delays, enhancing the overlap between communication and computation, and distributing the computational burden across model replicas during updates^[93] can effectively speed up the process. Moreover, for LLMs with extensive parameter sizes, a decentralized approach to memory assignment minimizes the memory overhead of redundant optimizer states, thereby optimizing memory usage on each accelerator.

Tensor parallelism operates by splitting computation tasks and aggregating results, but this process increases the communication burden between devices. Its optimization lies in improving the operator splitting method and enhancing communication efficiency. Matrix multiplication decomposition allows for the implementation of multi-dimensional tensor parallelism, exemplified by the Colossal-AI^[86] framework’s support for 1D^[17, 18], 2D^[94], 2.5D^[95], and 3D^[96] splits, alongside the ATP^[97] feature for automatic selection. It becomes particularly important to explore communication compression, optimize the pipeline between computation and communication, and enhance their overlap^[98, 99].

Pipeline parallelism inevitably creates resource-idle problems. The minimization of “bubble” effects, aimed at boosting overall efficiency, is addressed by techniques developed in Gpipe^[100], Megatron-LM^[17, 18], and Chimera^[105]. Concurrently, the challenge of memory imbalances, a significant concern in pipeline parallelism, is tackled by innovations presented in GEMS^[106], BPipe^[107], and Chimera^[105], as well as the DualPipe method proposed in DeepSeek V3^[6]. Moreover, applying pipeline parallelism strategies to dynamic networks enhances the performance of Transformer-based networks in neural architecture search (NAS)^[108, 127].

4.2.3 Job Partition

Job partition and scheduling, refer to deploying hybrid parallelism with an automatic or semi-auto-

Table 8. Enhancing Distributed Computing: From Parallel Optimization to Fault Tolerance

Category	Study	Highlight
DP, TP, PP Optimizations	Jia <i>et al.</i> , 2018 ^[92]	Hierarchical ALL-Reduce
	Xu <i>et al.</i> , 2020 ^[93]	Shard the weight update computation
	ZeRO ^[81]	Zero redundancy optimizer (ZeRO)
	FSDP ^[16]	Fully sharded data parallel (FSDP)
	Colossal-AI ^[86]	1D ^[17, 18] , 2D ^[94] , 2.5D ^[95] , and 3D ^[96] tensor parallelism
	ATP ^[97]	Adaptive tensor parallelism
	AccTFM ^[98]	Fine-grained computation and communication pipeline, communication compression
	Wang <i>et al.</i> , 2022 ^[99]	Overlap communication and computation via decomposition
	Gpipe ^[100]	Mini-batch, pipeline bubble reduction
	Megatron-LM ^[17, 18]	Interleaved pipeline, 1F1B
	DAPPLE ^[101]	Interleaved pipeline, dynamic programming
	PipeDream ^[102]	Async update
	PipeDream-2DW ^[103]	Async update, double-buffer
	HetPipe ^[104]	Wave synchronous parallel, parameter server
	Chimera ^[105]	Bidirectional pipelines, hyper-parameter space guidance
	GEMS ^[106]	Bidirectional pipelines, memory-aware synchronous training
	BPIPE ^[107]	Transfer activations to reduce memory imbalance across stages
	vPIPE ^[108]	NAS, re-balancing the layer distribution, dynamic programming
Job Partition & Scheduling	RaNNC ^[109]	Automatic graph partitioning (2D), dynamic programming
	Merak ^[87]	Automatic model partitioner (3D), auto graph-sharding algorithm
	Galvatron ^[110]	Automatic parallelism, decision tree, dynamic programming
	Alpa ^[111]	Inter-op: dynamic programming; intra-op: integer linear programming (ILP)
	Luo <i>et al.</i> , 2022 ^[112]	Pipeline partition and device mapping, dynamic programming
	Piper ^[113]	DAG modeling, dynamic programming
	Auto-MAP ^[114]	Works on HLO IR, Deep Q-Network
	Sia ^[115]	Scheduling for resource-adaptive training, ILP formulation
	CoCoNet ^[116]	DSL that contains computation and communication, scheduling improves overlapping
	Centauri ^[117]	Communication partition and hierarchical scheduling schemes or optimized overlap
Fault Tolerant	DeepFreeze ^[118]	Asynchronous I/O, lightweight serialization, sharding
	CheckFreq ^[119]	Fine-grained checkpointing in mini-batch
	Check-N-Run ^[120]	Differential checkpointing, quantized checkpointing
	Gemini ^[121]	In-memory checkpoints, mixed placement, checkpoint traffic scheduling
	Oobleck ^[89]	Elastic training, pipeline template
	Transom ^[122]	Pipeline fault tolerance, anomaly detection, asynchronous persistence
	Unicron ^[123]	Error detection, reduce downtime

matic model partitioner and scheduler when running LLMs. This approach is beneficial when running extensive models that leverage hundreds or thousands of accelerators, as it aims to enhance resource utilization during parallel training processes. A critical aspect of this strategy is the parallel interface provided by the programming frameworks, which should obviate the need for manual modifications to parallelize training. The simplicity significantly contributes to rapid implementation and eases the programmer's workload.

To address communication bandwidth bottle-

necks, strategic decisions based on the network topology of accelerators can be made using pre-defined, graph algorithms^[112, 128] to obtain a deployment order that is consistent with the specific communication architecture of the distributed system. Techniques such as grid search, complemented empirical tuning, are employed for hyper-parameter tuning, which is instrumental in balancing the computational load, memory usage, and communication overhead across accelerators, thereby enhancing the performance and efficiency of LLMs training.

In the realm of optimizing parallel execution for

^[16]<https://pytorch.org/blog/introducing-pytorch-fully-sharded-data-parallel-api/>, Jan. 2025.

models, beyond the application of predefined rules, a diverse array of strategies, including traditional algorithms such as dynamic programming, integer linear programming, and breadth-first search, offer guidance for automatic task division, and exploration of machine learning-based techniques is also feasible. Furthermore, enhancing the overlap between computational and communication tasks during parallel execution is important for efficiency.

4.2.4 Fault Tolerant

Fault tolerance refers to the ability of a system to continue functioning normally even when partial failures occur. This is particularly important in LLMs training, which is time-consuming and susceptible to software and hardware failures, potentially leading to significant time and financial losses.

Checkpointing is a widely used fault tolerance technique that saves the training state periodically for potential rollback. However, the considerable parameter size of LLMs leads to longer checkpoint access time, which presents a challenge to LLMs training. To address this, methods like differential checkpointing, quantization, and compression have been introduced to reduce checkpoint file sizes. Examples include CheckFreq^[119] and Check-N-Run^[120]. Additionally, memory hierarchy optimizations have been implemented. For instance, Gemini^[121] utilizes high-speed host memory and non-volatile memory express (NVMe) SSDs.

Elastic training, such as Oobleck^[89], Transom^[122],

and Unicorn^[123], focuses on improving resilient computation and facilitating rapid recovery from faults.

Redundant computations enhance the use of cost-effective computing resources (e.g., spot instances in cloud environments), as demonstrated by Varuna^[85] and Bamboo^[88].

4.3 Optimizing Memory Efficiency: Strategies from Recomputation to Offloading

In the training of LLMs, a significant challenge is meeting the demanding memory requirements within the constrained capacity of accelerators. In the forward propagation procedure of training, activation tensors are created and subsequently released after their utilization in the backward propagation procedure. This dynamic leads to a rise and fall in memory usage, potentially triggering out-of-memory (OOM) issues. The extensive parameter size characteristic of LLMs significantly amplifies this challenge.

To reduce peak memory usage without altering the data bit width, one approach is recomputation, which leverages additional computation in exchange for memory. Another approach is swapping tensors to off-accelerator memory to free accelerator memory. Offloading, unlike recomputation which is limited to the accelerator, uses host memory and SSDs to meet the extensive memory requirements of LLMs training, making it a technology worthy of attention. The corresponding papers are shown in Table 9, and the benchmarks for the majority of the methods include LLMs.

Table 9. Efficient Memory Management: Memory Usage Reduction and Corresponding Papers

Category	Study	Highlight
Recomputation	Chen <i>et al.</i> , 2016 ^[129]	Predefined periodic selection, static linear network
	Kusumoto <i>et al.</i> , 2019 ^[130]	Exact dynamic programming (DP), or approximate DP
	Checkmate ^[131]	Mixed integer linear program, two-phase LP rounding
	DTR ^[132]	Greedy online algorithm
	Korthikanti <i>et al.</i> , 2023 ^[133]	Predefined strategy in Transformer, selective activation recomputation
Offloading	vDNN ^[134]	Static policy (all or conv-only feature maps), greedy dynamic policy
	ZeRO-Offload ^[135]	Offloading data and computation to host memory and CPU
	ZeRO-Infinity ^[136]	Offloading to CPU or NVMe memory, memory-centric tiling for working memory
	PatrickStar ^[137]	Dynamic management based on life cycles, chunk eviction strategy
	G10 ^[138]	Based on semantic knowledge at compilation, page migration, unified space with SSD
Mixed	Capuchin ^[139]	Based on access patterns tracked at runtime, measured metrics
	KARMA ^[140]	Performance model, two-stage optimization, ILP interleaving layer swapping with redundant recompute
	STR ^[141]	Hybrid tensor re-generation strategy mixed ILP, constraint relaxation and approximation

4.3.1 Recomputation

Recomputation or rematerialization, often referred to as checkpointing, is an essential optimization technique in neural network training, especially for LLMs. In this approach, only the outputs of selected key layers are saved as checkpoints during the forward propagation phase, without retaining the activation values of all layers. These checkpoints are then utilized to recompute the required activation values during the backward propagation phase. As a technique widely applied during the training of DNN networks, recomputation was initially proposed by Chen *et al.*^[129]. After years of development, mainstream programming frameworks have built in this feature.

The main challenge of this technique is to strike a suitable balance between memory consumption and computational costs, which can be determined through various methods such as predefined setting, integer programming, dynamic programming, or greedy algorithms^[129–132]. These methods help determine which intermediate activation values should be kept.

Considering the network structure characteristics of LLMs, selective recomputation^[133] is particularly effective. This approach weighs the benefits of specific activation tensors, and focuses on recomputing activation values that require less computational effort but have a high number of tensor parameters. It reduces activation memory up to five times, and achieves 29% faster than normal recomputation on a 530B model^[133].

4.3.2 Offloading

Offloading, also known as memory swapping, is a technique used in heterogeneous computing systems to transfer data from accelerators to host memory. This strategy aims to alleviate the memory load on accelerators and enables the data to be reloaded into the accelerator as needed. With the evolution of DNN training, offloading technology has been introduced to meet diverse requirements^[142, 143], with vDNN^[134] being one of the pioneers in using host memory to relieve the memory burden on accelerators.

The main challenges in offloading include determining (i.e., what, where, when, and how) the optimal data to swap and managing efficient data transfer without substantially slowing down program execution. Offloading decisions can be made using a lot

of strategies, ranging from predefined settings to more complex algorithms such as greedy algorithms, genetic algorithms, or integer programming. Unlike recomputation, which only involves computation on accelerators, offloading also involves the management of host memory and data transfer, adding complexity to its implementation.

Considering the substantial memory requirements of LLMs training, it is essential to fully utilize all host-side resources. Offloading extends beyond just leveraging large-capacity host memory; it also encompasses the use of non-volatile memory options such as SSDs^[136, 138]. Offloading targets beyond just activation values, encompassing any data that can benefit from being in a heterogeneous system. Therefore, the gradient data can be transferred to the host side and utilized to update the gradient using idle CPU resources^[135], thereby enhancing resource utilization and flexibility. At the same time, due to the multiple times and large amount of data involved, offloading may conflict with other messaging within the server or computing system. Bad memory allocation can also reduce access efficiency. Therefore, optimizing transmission efficiency becomes one of the optimization directions^[137, 138]. In the benchmark that includes BERT, G10^[138] outperforms FlashNeuron by 1.56x and DeepUM+ by 1.31x, both of which are pre-LLMs-era methods.

4.3.3 Recomputation & Offloading

Integrating recomputation and offloading strategies presents a viable approach to significantly reducing memory requirements while preserving the training efficiency of neural networks. The principal challenge lies in accurately estimating the computational and memory costs associated with these strategies, as well as in making informed decisions regarding the optimal moments for tensor release and reconstruction. SuperNeurons^[144] is the first work to combine recomputation and offloading, making processing decisions for activation values based on operator types, but its static strategy approach cannot achieve the optimal solution. Follow-on studies have embraced a more dynamic approach, which considers both strategies. Runtime access patterns and advanced modelling techniques are used to accurately estimate the computational costs. This enables more advantageous decision-making processes when balancing between recomputation and offloading^[139–141].

4.4 Computation Enhancements: Quantization, Sparsity, and Kernel Optimization

Deep learning models, particularly in the realm of LLMs, are generally less sensitive to numerical precision and often incorporate redundant parameters. However, scheduling optimizations previously discussed (as referenced in Table 8 and Table 9) focus on efficient task allocation at the computational graph level, and do not fully exploit the aforementioned characteristics of deep learning models nor significantly changing the specific kernel computational behavior. Consequently, explorations in areas such as approximation, model compression, and transformation seek to simplify the computational aspects of models and enhance the efficiency of their kernels on specific accelerator platforms, while also maintaining accuracy. Specifically, techniques such as quantization and kernel optimization, as shown in Table 10, offer a balanced strategy to optimize computational efficiency in deep learning, particularly in LLMs, without compromising on model accuracy.

4.4.1 Quantization

Quantization refers to representing the numerical values of neural networks using low bit-widths (i.e., less than 32). This strategy can be employed to substitute the standard numerical values in model computations with lower bit-width approximations, lever-

aging cost-efficient approximate arithmetic operations for faster execution, considering the non-absolute accuracy of model outputs in deep learning. As a form of approximation, quantization is a popular optimization method that has been widely validated to reduce the bit-widths of numerical values while preserving acceptable accuracy loss.

Quantization training techniques can significantly improve training efficiency. This approach directly reduces the memory overhead of models and benefits from the enhanced computing capacity (i.e., FLOPS) of low bit-width operations^[6, 145–150]. For example, DeepSeek V3^[6] demonstrates effective FP8 training for large-scale LLMs, including fine-grained quantization with tile/block-wise grouping, high-precision accumulation, and low-precision activation caching and communication. Implementing low bit-width quantization can substantially lower the memory requirement of models, making it feasible to run models with larger parameter sizes on the same accelerator. Additionally, low bit-width operations have lower computational costs and reduced access to off-chip memory, making accelerators more energy-efficient. Moreover, since accelerators are more powerful in executing low bit-width operations, quantization can also elevate performance ceilings. Quantization training algorithms focus on solving the problem of accuracy degradation caused by low precision^[161, 162], while the joint optimization of hardware and software aims to better support quantization and further improve ener-

Table 10. Advances in Computational Efficiency: Techniques and Related Papers

Category	Study	Highlight
Quantization	Cambier <i>et al.</i> , 2020 ^[145]	Shifted and squeezed FP8 (S2FP8) training
	Zhang <i>et al.</i> , 2020 ^[146]	Adaptive fixed-point training
	FP8-LM ^[147]	FP8 automatic mixed-precision
	QLoRA ^[148]	Quantized Low Rank Adapter, finetuning of quantized LLMs
	Mesa ^[149] , GACT ^[150]	Activation Compressed Training (ACT)
	1-bit SGD ^[151]	Gradient compression
	1-bit Adam ^[152]	
	0/1 Adam ^[153]	
Kernel optimization	TASO ^[154]	Graph substitution
	Ivanov <i>et al.</i> , 2021 ^[155]	Data movement optimization
	Rabe and Staats, 2021 ^[156]	Reducing attention memory requirement
	FlashAttention ^[157]	Tiling, kernel fusion, I/O complexity analysis
	FlashAttention-2 ^[158]	Improving non-matmul FLOPs and thread block parallelism
	XLA	HLO IR, target-independent optimization, fusion
	TorchInductor	Define-by-Run IR, reordering, fusion and inlining
	Chimera ^[159]	Fused kernels
	Wang and Shen, 2023 ^[160]	Automatic kernel generation, reinforcement learning

gy efficiency^[163].

Besides quantizing computations, quantizing gradient (i.e., 1-bit SGD^[151], 1-bit Adam^[152], and 0/1 Adam^[153]) transmissions can directly cut down on data transfer amounts and reduce latency, thereby accelerating the overall training process.

4.4.2 Kernel Optimization

Kernel optimization refers to improving the performance of neural network operators through methods such as equivalent transformations and advanced compilation techniques. The efficiency of execution for algorithms designed for specific applications may not always align with optimal hardware resource utilization. Strategic transformations and kernel optimizations tailored to specific hardware platforms and computational contexts can significantly enhance its utilization. These strategies aim at assigning computational execution more efficiently, optimizing data movement, and ensuring hardware-aware efficient memory access. Such optimizations are critical for reducing idle time in hardware processing units and providing accelerators with more efficient operator implementations.

For memory-intensive operations like attention mechanisms, the memory footprint is reduced by FlashAttention^[157] through tiling and improvements to the softmax, and memory access efficiency is improved through the reduction of data movement^[155]. FlashAttention-2^[158] has further improved the efficiency of attention execution, achieving three to ten times faster than standard attention execution. Additionally, Kernel Fusion, an approach broadly adopted in the DNN era, works by fusing multiple operators, thereby reducing off-chip memory access and enhancing the efficiency of processors in executing computations for LLMs^[157].

Efficient operator kernels can be sourced from high-performance libraries (NVIDIA cuDNN, Cambricon CNL, etc.), or implemented by deep learning compilers (such as Glow, nvFuser, XLA, TorchInductor). High-performance libraries are designed to accelerate deep learning computations on specific hardware platforms, with a lot of hand-written implementations and optimizations which are provided by hardware vendors. As for compilers, they are capable of identifying more efficient methods for operator implementation, either through predefined rules or search algorithms, ensuring that these operators

maintain compatibility across different platforms.

4.5 Comparison of Training and Inference

As discussed in Subsections 4.2 to 4.4, we have introduced and discussed the challenges associated with the model training process and the strategies for optimization. By contrasting optimization techniques used in the inference process, we illustrate that inference requires a less complex computing system, thereby underscoring that AI computing systems for LLMs training is more intricate. Then we briefly explain several unique optimization strategies for inference, consciously avoiding the repetition of methods that are applicable to both training and inference, as the focus of this paper is still on LLMs training.

The distinctions between inference and training computations are primarily categorized into three folds.

1) *Simplified Computing Systems.* It is possible to efficiently conduct inference tasks of LLMs on a single accelerator by incorporating techniques such as quantization, pruning, and sparsity. When combined with tensor or pipeline parallelism, even super-large models can be inferred on a single computing node. From a computational system perspective, inference operations require less hardware infrastructure and are less complex compared with distributed training.

2) *Low Memory Requirements.* The inference process of an LLM only involves forward propagation and does not require backward propagation. This means there is no necessity to store activation values from the forward pass, nor is there a need for gradient updates or maintaining optimizer states, significantly reducing the memory resources required compared with training.

3) *Data Precision Insensitivity.* Moreover, since there is no need to worry about the numerical precision issues of backward propagation, more aggressive quantization and model pruning can be adopted to target the model redundancy and further boost the efficiency of inference.

For inference on large models, most of the optimization methods used on training are also applicable, including quantization, pruning, sparse, fusion, and compilation optimization^[164–166]. Additionally, there are some unique optimizations for the inference of LLMs, including function approximation, key-value (KV) cache, variable length optimization, and sampling optimization.

- Function approximation involves replacing the

exact value of a non-linear function with an approximate value that is easier and less resource-intensive for hardware to compute. Most of the work focuses on the softmax computation in attention because of its non-linearity, high frequency, and poor parallelism, which acts as a bottleneck. Specifically, the work includes using interpolation functions, polynomial fitting, and altering the base of the exponential function in softmax from the natural base e to 2^[65, 68, 167].

- KV cache refers to storing the previously computed Key and Value pairs from the scaled dot-product within the attention mechanism. While this reuse avoids redundant computations during token generation, it incurs significant memory overhead. To mitigate this, various optimization techniques have been proposed, including memory-efficient management^[168], importance-based pruning^[169], adaptive compression^[170], and low-rank projection methods such as DeepSeek’s Multi-head Latent Attention (MLA)^[6]. These approaches collectively address the trade-off between computation and memory through data-saving optimizations and compression strategies.

- Variable length addresses the challenge of altering tensor shapes in computational graphs, due to varying input and output sequence lengths. The padding used in batch processing can lead to inefficient and uneven resource usage. Packing and continuous batching are employed to minimize the impact of ineffective padding^[171–173]. At the same time, compilation optimization targeting variable length is also a worthwhile method to explore^[174, 175].

- Sampling optimization includes token decoding sampling, as well as speculative sampling of large and small models. These methods affect the quality of the text generated by the LLM inference. Token decoding sampling refers to the method of selecting generated tokens, including greedy, top- k , nucleus sampling, random sampling, temperature sampling, and beam search. Speculative sampling utilizes lower-cost smaller models to perform multiple predictions and samplings, which are then scored by a larger model for evaluation. This strategy effectively reduces the high-computational demands of token-by-token generation in higher-cost models, thereby improving the overall inference efficiency.

For a deeper understanding of LLMs inference, you can refer to the related surveys^[22, 23].

5 Challenge and Future Work

In the preceding sections, this paper provided an

overview of AI computing systems for LLMs training from perspectives such as algorithms, hardware, and system software. Each area faces challenges but also presents significant opportunities. Overall, there is a need for more efficient algorithmic designs, hardware platforms co-designed with software, and enhanced task scheduling strategies.

Improved Algorithm. In terms of both operational intensity analysis and actual measured data^[65–68], the attention mechanism continues to be a computational challenge in each block within the current Transformer architecture. Despite various attempts at efficiency improvements (e.g., kernel optimization, algorithmic changes, computational approximation, and sparsity techniques), the attention mechanism remains fundamentally limited by memory constraints due to its inherent low computational density. To fundamentally improve hardware efficiency and minimize downtime of computational units, it is necessary to further reduce reliance on the traditional attention mechanism. Investigating new forms of attention and their substitutes^[176–178] is one approach. Another is creating modules with higher computational density to compute sequence correlations in place of attention. Exploring architectures that are devoid of attention mechanisms or reducing the role of attention computations could also significantly strengthen the overall system.

Hardware-Software Co-Design. Although widely-used hardware continues to develop, optimization efforts frequently overlook the unique requirements of LLMs (e.g., accelerate softmax). Currently, research on optimizing LLMs focuses on memory computation, quantization, and sparsity, aiming to accelerate through hardware view or software-hardware co-optimization. However, due to limited commercial support, the development of these accelerators is mostly confined to academic research. Therefore, it is imperative to explore software and hardware co-optimization methods in a real industrial environment, involving co-designing computing nodes optimized for distributed environments, accelerators targeting computational bottlenecks, and co-designing compilers to maximize hardware resource. For example, dynamic bit-width computation could be achieved by leveraging the hardware low bit-width quantization features and exploring dynamic range management and fine-grained partitioning.

Efficient Scheduling. As model sizes, datasets, and text lengths expand, the demand for resources during the training process significantly increases. The trend

suggests that training future-generation models requires the coordinated effort of thousands of accelerators for distributed training, resulting in extensive data transfers across accelerators. Whether it is data parallelism, tensor parallelism, or pipeline parallelism, all lead to substantial ALL-Reduce communication. The use of offloading techniques and recomputation strategies further increases data movements. Considering the diversity of training environments (e.g., the mixed use of accelerators, bandwidth constraints, and blocking network topologies), we believe it is necessary to explore global-info-aware scheduling strategies. This exploration involves introducing heterogeneous-aware computation to optimize the utilization of CPU and accelerator resources, developing topology-aware communication strategies^[179] to reduce the impact of communication bottlenecks, and pursuing efficient scheduling techniques to overlap computation and communication^[117]. At the same time, in order to mitigate the impact of local failures on the training process, the dynamic adjustment mechanism can improve the training efficiency and system robustness.

The development of LLMs training requires not only isolated advancements but also systemic optimizations that span across algorithms, hardware-software co-design, and efficient scheduling strategies. DeepSeek V3 demonstrates the potential of integrated frameworks through coordinated innovations: efficient MLA attention, hardware-accelerated communication enabled by dedicated SMs, enhanced FP8 training, and load-balanced MoE. These advancements validate the proposed future directions, underscoring the importance of multidimensional synergy. The open accessibility of such systematic work provides valuable insights to advance the research and deployment of future AI computing systems.

6 Conclusions

In this paper, we provided a systematic exploration of the AI computational systems for LLMs training, with details in the computational processes involved. We discussed the algorithm structures and computing patterns of LLMs training, alongside a review of the current hardware platforms. Considering the significant computational and memory requirements imposed by extensive parameters of models, against the backdrop of limited available hardware resources, our discussion extended to distributed paral-

lel approaches and a suite of optimization strategies, which aim at enhancing the feasibility and efficiency of running LLMs.

This survey offers a comprehensive overview of modern AI computing systems for LLMs training, focusing on their algorithms, software and hardware infrastructure, as well as the operational processes. We hope that this paper not only offers insightful guidance but also serves as a catalyst for further investigation into the advanced AI infrastructure solutions for LLMs.

Acknowledgements We thank Jian-Xing Xu for his timely assistance during the revision of this paper. We are grateful to the reviewers for their constructive suggestions.

Conflict of Interest The authors declare that they have no conflict of interest.

References

- [1] Vaswani A, Shazeer N, Parmar N, Uszkoreit J, Jones L, Gomez A N, Kaiser Ł, Polosukhin I. Attention is all you need. In *Proc. the 31st International Conference on Neural Information Processing Systems*, Dec. 2017, pp.6000–6010.
- [2] Devlin J, Chang M W, Lee K, Toutanova K. BERT: Pre-training of deep bidirectional transformers for language understanding. In *Proc. the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, Jun. 2018, pp.4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423).
- [3] Radford A, Wu J, Child R, Luan D, Amodei D, Sutskever I. Language models are unsupervised multitask learners. *OpenAI Blog*, 2019, 1(8): Article No. 9.
- [4] Brown T B, Mann B, Ryder N *et al.* Language models are few-shot learners. In *Proc. the 34th International Conference on Neural Information Processing Systems*, Dec. 2020, Article No. 159.
- [5] Liu A, Feng B, Wang B *et al.* Deepseek-v2: A strong, economical, and efficient mixture-of-experts language model. arXiv: 2405.04434, 2024. <https://arxiv.org/abs/2405.04434>, Jan. 2025.
- [6] Liu A, Feng B, Xue B *et al.* Deepseek-v3 technical report. arXiv: 2412.19437, 2024. <https://arxiv.org/abs/2412.19437>, Jan. 2025.
- [7] Chen Y, Li L, Li W, Guo Q, Du Z, Xu Z. AI Computing Systems: An Application-Driven Perspective. Elsevier, 2023. DOI: [10.1016/C2021-0-02950-3](https://doi.org/10.1016/C2021-0-02950-3).
- [8] Chen T, Du Z, Sun N, Wang J, Wu C, Chen Y, Temam O. DianNao: A small-footprint high-throughput accelerator for ubiquitous machine-learning. In *Proc. the 19th International Conference on Architectural Support for*

- Programming Languages and Operating systems*, Mar. 2014, pp.269–284. DOI: [10.1145/2541940.2541967](https://doi.org/10.1145/2541940.2541967).
- [9] Chen Y, Luo T, Liu S, Zhang S, He L, Wang J, Li L, Chen T, Xu Z, Sun N, Temam O. DaDianNao: A machine-learning supercomputer. In *Proc. the 47th Annual IEEE/ACM International Symposium on Microarchitecture*, Dec. 2014, pp.609–622. DOI: [10.1109/MICRO.2014.58](https://doi.org/10.1109/MICRO.2014.58).
 - [10] Liu D, Chen T, Liu S, Zhou J, Zhou S, Teman O, Feng X, Zhou X, Chen Y. PuDianNao: A polyvalent machine learning accelerator. In *Proc. the 20th International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2015, pp.369–381. DOI: [10.1145/2694344.2694358](https://doi.org/10.1145/2694344.2694358).
 - [11] Du Z, Fasthuber R, Chen T, Ienne P, Li L, Luo T, Feng X, Chen Y, Temam O. ShiDianNao: Shifting vision processing closer to the sensor. In *Proc. the 42nd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2015, pp.92–104. DOI: [10.1145/2749469.2750389](https://doi.org/10.1145/2749469.2750389).
 - [12] Jouppi N P, Young C, Patil N *et al.* In-datacenter performance analysis of a tensor processing unit. In *Proc. the 44th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2017, pp.1–12. DOI: [10.1145/3079856.3080246](https://doi.org/10.1145/3079856.3080246).
 - [13] Rasley J, Rajbhandari S, Ruwase O, He Y. DeepSpeed: System optimizations enable training deep learning models with over 100 billion parameters. In *Proc. the 26th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Jul. 2020, pp.3505–3506. DOI: [10.1145/3394486.3406703](https://doi.org/10.1145/3394486.3406703).
 - [14] Li C, Yao Z, Wu X, Zhang M, Holmes C, Li C, He Y. DeepSpeed data efficiency: Improving deep learning model quality and training efficiency via efficient data sampling and routing. In *Proc. the 38th AAAI Conference on Artificial Intelligence*, Feb. 2024, pp.18490–18498. DOI: [10.1609/aaai.v38i16.29810](https://doi.org/10.1609/aaai.v38i16.29810).
 - [15] Aminabadi R Y, Rajbhandari S, Awan A A, Li C, Li D, Zheng E, Ruwase O, Smith S, Zhang M, Rasley J, He Y. DeepSpeed-inference: Enabling efficient inference of transformer models at unprecedented scale. In *Proc. the 2022 International Conference on High Performance Computing, Networking, Storage and Analysis*, Nov. 2022, Article No. 46.
 - [16] Rajbhandari S, Li C, Yao Z, Zhang M, Aminabadi R Y, Awan A A, Rasley J, He Y. DeepSpeed-MoE: Advancing mixture-of-experts inference and training to power next-generation AI scale. In *Proc. the 39th International Conference on Machine Learning*, Jul. 2022, pp.18332–18346.
 - [17] Shoenybi M, Patwary M, Puri R, LeGresley P, Casper J, Catanzaro B. Megatron-LM: Training multi-billion parameter language models using model parallelism. arXiv: 1909.08053, 2019. <https://arxiv.org/abs/1909.08053>, Nov. 2024.
 - [18] Narayanan D, Shoenybi M, Casper J *et al.* Efficient large-scale language model training on GPU clusters using megatron-LM. In *Proc. the 2021 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2021.
 - [19] Lin T, Wang Y, Liu X, Qiu X. A survey of transformers. *AI Open*, 2022, 3: 111–132. DOI: [10.1016/j.aiopen.2022.10.001](https://doi.org/10.1016/j.aiopen.2022.10.001).
 - [20] Zhao W X, Zhou K, Li J, Tang T, Wang X, Hou Y, Min Y, Zhang B, Zhang J, Dong Z, Du Y, Yang C, Chen Y, Chen Z, Jiang J, Ren R, Li Y, Tang X, Liu Z, Liu P, Nie J Y, Wen J R. A survey of large language models. arXiv: 2303.18223, 2023. <https://arxiv.org/abs/2303.18223>, Nov. 2024.
 - [21] Wu J, Gan W, Chen Z, Wan S, Philip S Y. Multimodal large language models: A survey. In *Proc. the 2023 IEEE International Conference on Big Data (BigData)*, Dec. 2023, pp.2247–2256. DOI: [10.1109/BigData59044.2023.10386743](https://doi.org/10.1109/BigData59044.2023.10386743).
 - [22] Kim S, Hooper C, Wattanawong T, Kang M, Yan R, Genc H, Dinh G, Huang Q, Keutner K, Mahoney M W, Shao Y S, Gholami A. Full stack optimization of transformer inference: A survey. arXiv: 2302.14017, 2023. <https://arxiv.org/abs/2302.14017>, Nov. 2024.
 - [23] Miao X, Oliaro G, Zhang Z, Cheng X, Jin H, Chen T, Jia Z. Towards efficient generative large language model serving: A survey from algorithms to systems. arXiv: 2312.15234, 2023. <https://arxiv.org/abs/2312.15234>, Nov. 2024.
 - [24] Shen L, Sun Y, Yu Z, Ding L, Tian X, Tao D. On efficient training of large-scale deep learning models: A literature review. arXiv: 2304.03589, 2023. <https://arxiv.org/abs/2304.03589>, Nov. 2024.
 - [25] Zhuang B, Liu J, Pan Z, He H, Weng Y, Shen C. A survey on efficient training of transformers. In *Proc. the 32nd International Joint Conference on Artificial Intelligence*, Aug. 2023, pp.6823–6831. DOI: [10.24963/ijcai.2023/764](https://doi.org/10.24963/ijcai.2023/764).
 - [26] Raffel C, Shazeer N, Roberts A, Lee K, Narang S, Matena M, Zhou Y, Li W, Liu P J. Exploring the limits of transfer learning with a unified text-to-text transformer. *The Journal of Machine Learning Research*, 2020, 21(1): 140.
 - [27] OpenAI. GPT-4 technical report. arXiv: 2303.08774, 2023. <https://arxiv.org/abs/2303.08774>, Nov. 2024.
 - [28] Sun Y, Wang S, Feng S, Ding S, Pang C, Shang J, Liu J, Chen X, Zhao Y, Lu Y, Liu W, Wu Z, Gong W, Liang J, Shang Z, Sun P, Liu W, Ouyang X, Yu D, Tian H, Wu H, Wang H. Ernie 3.0: Large-scale knowledge enhanced pre-training for language understanding and generation. arXiv: 2107.02137, 2021. <https://arxiv.org/abs/2107.02137>, Nov. 2024.
 - [29] Rae J W, Borgeaud S, Cai T *et al.* Scaling language models: Methods, analysis & insights from training gopher. arXiv: 2112.11446, 2021. <https://arxiv.org/abs/2112.11446>, Nov. 2024.
 - [30] Chowdhery A, Narang S, Devlin J *et al.* PaLM: Scaling language modeling with pathways. *The Journal of Machine Learning Research*, 2023, 24(1): 240.

- [31] Zhang S, Roller S, Goyal N *et al.* OPT: Open pre-trained transformer language models. arXiv: 2205.01068, 2022. <https://arxiv.org/abs/2205.01068>, Nov. 2024.
- [32] Scao TL, Fan A, Akiki C *et al.* BLOOM: A 176b-parameter open-access multilingual language model. arXiv: 2211.05100, 2022. <https://arxiv.org/abs/2211.05100>, Nov. 2024.
- [33] Du Z, Qian Y, Liu X, Ding M, Qiu J, Yang Z, Tang J. GLM: General language model pretraining with autoregressive blank infilling. In *Proc. the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, May 2022, pp.320–335. DOI: [10.18653/v1/2022.acl-long.26](https://doi.org/10.18653/v1/2022.acl-long.26).
- [34] Touvron H, Lavril T, Izacard G, Martinet X, Lachaux M A, Lacroix T, Rozière B, Goyal N, Hambro E, Azhar F, Rodriguez A, Joulin A, Grave E, Lample G. LLaMA: Open and efficient foundation language models. arXiv: 2302.13971, 2023. <https://arxiv.org/abs/2302.13971>, Nov. 2024.
- [35] Ren X, Zhou P, Meng X, Huang X, Wang Y, Wang W, Li P, Zhang X, Podolskiy A, Arshinov G, Bout A, Piontkovskaya I, Wei J, Jiang X, Su T, Liu Q, Yao J. PanGu- Σ : Towards trillion parameter language model with sparse heterogeneous computing. arXiv: 2303.10845, 2023. <https://arxiv.org/abs/2303.10845>, Nov. 2024.
- [36] Touvron H, Martin L, Stone K *et al.* Llama 2: Open foundation and fine-tuned chat models. arXiv: 2307.09288, 2023. <https://arxiv.org/abs/2307.09288>, Nov. 2024.
- [37] Li Z, Yang B, Liu Q, Ma Z, Zhang S, Yang J, Sun Y, Liu Y, Bai X. Monkey: Image resolution and text label are important things for large multi-modal models. In *Proc. the 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition*, Jun. 2024, pp.26753–26763. DOI: [10.1109/CVPR52733.2024.02527](https://doi.org/10.1109/CVPR52733.2024.02527).
- [38] Anil R, Borgeand S, Alayrac JB, *et al.* Gemini: A family of highly capable multimodal models. arXiv: 2312.11805, 2023. <https://arxiv.org/abs/2312.11805>, Nov. 2024.
- [39] Bai J, Bai S, Chu Y *et al.* Qwen technical report. arXiv: 2309.16609, 2023. <https://arxiv.org/abs/2309.16609>, Nov. 2024.
- [40] Kaplan J, McCandlish S, Henighan T, Brown T B, Chess B, Child R, Gray S, Radford A, Wu J, Amodei D. Scaling laws for neural language models. arXiv: 2001.08361, 2020. <https://arxiv.org/abs/2001.08361>, Nov. 2024.
- [41] Du N, Huang Y, Dai A M *et al.* GLaM: Efficient scaling of language models with mixture-of-experts. In *Proc. the 39th International Conference on Machine Learning*, Jul. 2022, pp.5547–5569.
- [42] Fedus W, Zoph B, Shazeer N. Switch transformers: Scaling to trillion parameter models with simple and efficient sparsity. *The Journal of Machine Learning Research*, 2022, 23(1): 120.
- [43] Dai D, Deng C, Zhao C *et al.* Deepseekmoe: Towards ultimate expert specialization in mixture-of-experts language models. arXiv: 2401.06066, 2024. <https://arxiv.org/abs/2401.06066>, Jan. 2025.
- [44] Child R, Gray S, Radford A, Sutskever I. Generating long sequences with sparse transformers. arXiv: 1904.10509, 2019. <https://arxiv.org/abs/1904.10509>, Nov. 2024.
- [45] Kitaev N, Kaiser L, Levskaya A. Reformer: The efficient transformer. In *Proc. the 8th International Conference on Learning Representations*, Apr. 2020.
- [46] Beltagy I, Peters M E, Cohan A. Longformer: The long-document transformer. arXiv: 2004.05150, 2020. <https://arxiv.org/abs/2004.05150>, Nov. 2024.
- [47] Zaheer M, Guruganesh G, Dubey A, Ainslie J, Alberti C, Ontanon S, Pham P, Ravula A, Wang Q, Yang L, Ahmed A. Big bird: Transformers for longer sequences. In *Proc. the 34th International Conference on Neural Information Processing Systems*, Dec. 2020, Article No. 1450.
- [48] Xiao G, Tian Y, Chen B, Han S, Lewis M. Efficient streaming language models with attention sinks. In *Proc. the 12th International Conference on Learning Representations*, May 2024.
- [49] Ratner N, Levine Y, Belinkov Y *et al.* Parallel context windows for large language models. In *Proc. the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, Jul. 2023, pp.6383–6402. DOI: [10.18653/v1/2023.acl-long.352](https://doi.org/10.18653/v1/2023.acl-long.352).
- [50] Ding J, Ma S, Dong L, Zhang X, Huang S, Wang W, Zheng N, Wei F. LongNet: Scaling transformers to 1, 000, 000, 000 tokens. arXiv: 2307.02486, 2023. <https://arxiv.org/abs/2307.02486>, Nov. 2024.
- [51] Tay Y, Dehghani M, Bahri D, Metzler D. Efficient transformers: A survey. *ACM Computing Surveys*, 2023, 55(6): 109. DOI: [10.1145/3530811](https://doi.org/10.1145/3530811).
- [52] Lewis M, Liu Y, Goyal N, Ghazvininejad M, Mohamed A, Levy O, Stoyanov V, Zettlemoyer L. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proc. the 58th Annual Meeting of the Association for Computational Linguistics*, Jul. 2020, pp.7871–7880. DOI: [10.18653/v1/2020.acl-main.703](https://doi.org/10.18653/v1/2020.acl-main.703).
- [53] Xue L, Constant N, Roberts A, Kale M, Al-Rfou R, Siddhant A, Barua A, Raffel C. mT5: A massively multilingual pre-trained text-to-text transformer. In *Proc. the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, Jun. 2021, pp.483–498. DOI: [10.18653/v1/2021.naacl-main.41](https://doi.org/10.18653/v1/2021.naacl-main.41).
- [54] Tay Y, Dehghani M, Tran V Q, Garcia X, Wei J, Wang X, Chung H W, Bahri D, Schuster T, Zheng S, Zhou D, Houlisby N, Metzler D. UL2: Unifying language learning paradigms. In *Proc. the Eleventh International Conference on Learning Representations*, May 2023.
- [55] Wang Y, Le H, Gotmare A, Bui N D, Li J, Hoi S C. CodeT5+: Open code large language models for code understanding and generation. In *Proc. the 2023 Conference on Empirical Methods in Natural Language Processing*, Dec. 2023, pp.1069–1088. DOI: [10.18653/v1/2023.emnlp-main.68](https://doi.org/10.18653/v1/2023.emnlp-main.68).

- [56] Soltan S, Ananthakrishnan S, FitzGerald J, Gupta R, Hamza W, Khan H, Peris C, Rawls S, Rosenbaum A, Rumshisky A, Prakash C S, Sridhar M, Triefenbach F, Verma A, Tur G, Natarajan P. AlexaTM 20B: Few-shot learning using a large-scale multilingual seq2seq model. arXiv: 2208.01448, 2022. <https://arxiv.org/abs/2208.01448>, Nov. 2024.
- [57] Hu E J, Shen Y, Wallis P, Allen-Zhu Z, Li Y, Wang S, Wang L, Chen W. LoRA: Low-rank adaptation of large language models. In *Proc. the 10th International Conference on Learning Representations*, Apr. 2021.
- [58] Li X L, Liang P. Prefix-tuning: Optimizing continuous prompts for generation. In *Proc. the 59th Annual Meeting of the Association for Computational Linguistics and the 11th International Joint Conference on Natural Language Processing*, Aug. 2021, pp.4582–4597. DOI: [10.18653/v1/2021.acl-long.353](https://doi.org/10.18653/v1/2021.acl-long.353).
- [59] Hu Z, Wang L, Lan Y, Xu W, Lim E P, Bing L, Xu X, Poria S, Lee R. LLM-adapters: An adapter family for parameter-efficient fine-tuning of large language models. In *Proc. the 2023 Conference on Empirical Methods in Natural Language Processing*, Dec. 2023, pp.5254–5276. DOI: [10.18653/v1/2023.emnlp-main.319](https://doi.org/10.18653/v1/2023.emnlp-main.319).
- [60] Wei J, Bosma M, Zhao V Y, Guu K, Yu A W, Lester B, Du N, Dai A M, Le Q V. Finetuned language models are zero-shot learners. In *Proc. the 10th International Conference on Learning Representations*, Apr. 2022.
- [61] Wei J, Wang X, Schuurmans D, Bosma M, Ichter B, Xia F, Chi E H, Le Q V, Zhou D. Chain-of-thought prompting elicits reasoning in large language models. In *Proc. the 36th International Conference on Neural Information Processing Systems*, Nov. 2022, Article No. 1800.
- [62] Ouyang L, Wu J, Jiang X et al. Training language models to follow instructions with human feedback. In *Proc. the 36th International Conference on Neural Information Processing Systems*, Nov. 2022, Article No. 2011.
- [63] Yu T, Zhu H. Hyper-parameter optimization: A review of algorithms and applications. arXiv: 2003.05689, 2020. <https://arxiv.org/abs/2003.05689>, Nov. 2024.
- [64] Williams R J, Zipser D. A learning algorithm for continually running fully recurrent neural networks. *Neural Computation*, 1989, 1(2): 270–280. 10.1162/neco.1989.1.2.270.
- [65] Wang H, Zhang Z, Han S. SpAtten: Efficient sparse attention architecture with cascade token and head pruning. In *Proc. the 2021 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb. 27-Mar. 3, 2021, pp.97–110. DOI: [10.1109/HPCA51647.2021.00018](https://doi.org/10.1109/HPCA51647.2021.00018).
- [66] Zeng W, Li M, Xiong W, Tong T, Lu W J, Tan J, Wang R, Huang R. MPCViT: Searching for accurate and efficient MPC-friendly vision transformer with heterogeneous attention. In *Proc. the 2023 IEEE/CVF International Conference on Computer Vision*, Oct. 2023, pp.5052–5063. DOI: [10.1109/ICCV51070.2023.00466](https://doi.org/10.1109/ICCV51070.2023.00466).
- [67] Liu L, Qu Z, Chen Z, Ding Y, Xie Y. Transformer acceleration with dynamic sparse attention. arXiv: 2110.11299, 2021. <https://arxiv.org/abs/2110.11299>, Nov. 2024.
- [68] Stevens J R, Venkatesan R, Dai S, Khailany B, Raghunathan A. Softmax: Hardware/software co-design of an efficient softmax for transformers. In *Proc. the 58th ACM/IEEE Design Automation Conference (DAC)*, Dec. 2021, pp.469–474. DOI: [10.1109/DAC18074.2021.9586134](https://doi.org/10.1109/DAC18074.2021.9586134).
- [69] Liu S, Du Z, Tao J, Han D, Luo T, Xie Y, Chen Y, Chen T. Cambricon: An instruction set architecture for neural networks. In *Proc. the 43rd ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2016, pp.393–405. DOI: [10.1109/ISCA.2016.42](https://doi.org/10.1109/ISCA.2016.42).
- [70] Liu K, Jiang Z, Zhang J, Wei H, Zhong X, Tan L, Pan T, Huang T. Hostping: Diagnosing intra-host network bottlenecks in RDMA servers. In *Proc. the 20th USENIX Symposium on Networked Systems Design and Implementation*, Apr. 2023, pp.15–29.
- [71] Li A, Song S L, Chen J, Li J, Liu X, Tallent N R, Barker K J. Evaluating modern GPU interconnect: PCIe, NVLink, NV-SLI, NVSwitch and GPUDirect. *IEEE Trans. Parallel and Distributed Systems*, 2020, 31(1): 94–110. DOI: [10.1109/TPDS.2019.2928289](https://doi.org/10.1109/TPDS.2019.2928289).
- [72] Vazhkudai S S, De Supinski B R, Bland A S et al. The design, deployment, and evaluation of the CORAL preexascale systems. In *Proc. the 2018 International Conference for High Performance Computing, Networking, Storage and Analysis (SC)*, Nov. 2018, pp.661–672. DOI: [10.1109/SC.2018.00055](https://doi.org/10.1109/SC.2018.00055).
- [73] Sharma D D. Compute express link®: An open industry-standard interconnect enabling heterogeneous data-centric computing. In *Proc. the 2022 IEEE Symposium on High-Performance Interconnects (HOTI)*, Aug. 2022, pp.5–12. DOI: [10.1109/HOTI55740.2022.00017](https://doi.org/10.1109/HOTI55740.2022.00017).
- [74] Qi H, Dai L, Chen W, Jia Z, Lu X. Performance characterization of large language models on high-speed interconnects. In *Proc. the 2023 IEEE Symposium on High-Performance Interconnects (HOTI)*, Aug. 2023, pp.53–60. DOI: [10.1109/HOTI59126.2023.00022](https://doi.org/10.1109/HOTI59126.2023.00022).
- [75] Jouppi N, Kurian G, Li S, Ma P, Nagarajan R, Nai L, Patil N, Subramanian S, Swing A, Towles B, Young C, Zhou X, Zhou Z, Patterson D A. TPU v4: An optically reconfigurable supercomputer for machine learning with hardware support for embeddings. In *Proc. the 50th Annual International Symposium on Computer Architecture*, Jun. 2023, Article No. 82. DOI: [10.1145/3579371.3589350](https://doi.org/10.1145/3579371.3589350).
- [76] Jiang Z, Lin H, Zhong Y et al. Megascale: Scaling large language model training to more than 10000 GPUs. In *Proc. the 21st USENIX Symposium on Networked Systems Design and Implementation*, Apr. 2024, pp.745–760.
- [77] Naumov M, Kim J, Mudigere D et al. Deep learning training in facebook data centers: Design of scale-up and scale-out systems. arXiv: 2003.09518, 2020. <https://arxiv.org/abs/2003.09518>, Nov. 2024.
- [78] Tokui S, Okuta R, Akiba T, Niitani Y, Ogawa T, Saito S, Suzuki S, Uenishi K, Vogel B, Yamazaki Vincent H. Chainer: A deep learning framework for accelerating the

- research cycle. In *Proc. the 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, Aug. 2019, pp.2002–2011. DOI: [10.1145/3292500.3330756](https://doi.org/10.1145/3292500.3330756).
- [79] Shazeer N, Cheng Y, Parmar N *et al.* Mesh-TensorFlow: Deep learning for supercomputers. In *Proc. the 32nd International Conference on Neural Information Processing Systems*, Dec. 2018, pp.10435–10444.
- [80] Kim C, Lee H, Jeong M, Baek W, Yoon B, Kim I, Lim S, Kim S. torchpipe: On-the-fly pipeline parallelism for training giant models. arXiv: 2004.09910, 2020. <https://arxiv.org/abs/2004.09910>, Nov. 2024.
- [81] Rajbhandari S, Rasley J, Ruwase O, He Y. ZeRO: Memory optimizations toward training trillion parameter models. In *Proc. the 2020 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2020, Article No. 20.
- [82] Yuan J, Li X, Cheng C, Liu J, Guo R, Cai S, Yao C, Yang F, Yi X, Wu C, Zhang H, Zhao J. OneFlow: Redesign the distributed deep learning framework from scratch. arXiv: 2110.15032, 2021. <https://arxiv.org/abs/2110.15032>, Nov. 2024.
- [83] Jia X, Jiang L, Wang A, Xiao W, Shi Z, Zhang J, Li X, Chen L, Li Y, Zheng Z, Liu X, Lin W. Whale: Efficient giant model training over heterogeneous GPUs. In *Proc. the 2022 USENIX Annual Technical Conference*, Jul. 2022, pp.673–688.
- [84] Singh S, Bhatele A. AxoNN: An asynchronous, message-driven parallel framework for extreme-scale deep learning. In *Proc. the 2022 IEEE International Parallel and Distributed Processing Symposium*, May 30–Jun. 3, 2022, pp.606–616. DOI: [10.1109/IPDPS53621.2022.00065](https://doi.org/10.1109/IPDPS53621.2022.00065).
- [85] Athlur S, Saran N, Sivathanu M, Ramjee R, Kwatra N. Varuna: Scalable, low-cost training of massive deep learning models. In *Proc. the 17th European Conference on Computer Systems*, Apr. 2022, pp.472–487. DOI: [10.1145/3492321.3519584](https://doi.org/10.1145/3492321.3519584).
- [86] Li S, Liu H, Bian Z, Fang J, Huang H, Liu Y, Wang B, You Y. Colossal-AI: A unified deep learning system for large-scale parallel training. In *Proc. the 52nd International Conference on Parallel Processing*, Aug. 2023, pp.766–775. DOI: [10.1145/3605573.3605613](https://doi.org/10.1145/3605573.3605613).
- [87] Lai Z, Li S, Tang X, Ge K, Liu W, Duan Y, Qiao L, Li D. Merak: An efficient distributed DNN training framework with automated 3D parallelism for giant foundation models. *IEEE Trans. Parallel and Distributed Systems*, 2023, 34(5): 1466–1478. DOI: [10.1109/TPDS.2023.3247001](https://doi.org/10.1109/TPDS.2023.3247001).
- [88] Thorpe J, Zhao P, Eyolfson J, Qiao Y, Jia Z, Zhang M, Netravali R, Xu G H. Bamboo: Making preemptible instances resilient for affordable training of large DNNs. In *Proc. the 20th USENIX Symposium on Networked Systems Design and Implementation*, Apr. 2023, pp.497–513.
- [89] Jang I, Yang Z, Zhang Z, Jin X, Chowdhury M. Oobleck: Resilient distributed training of large models using pipeline templates. In *Proc. the 29th Symposium on Operating Systems Principles*, Oct. 2023, pp.382–395. DOI: [10.1145/3600006.3613152](https://doi.org/10.1145/3600006.3613152).
- [90] Lepikhin D, Lee H, Xu Y *et al.* Gshard: Scaling giant models with conditional computation and automatic sharding. arXiv:2006.16668, 2025. <https://arxiv.org/abs/2006.16668>, Jan. 2025.
- [91] Li S, Xue F, Baranwal C, Li Y, You Y. Sequence parallelism: Long sequence training from system perspective. arXiv: 2105.13120, 2021. <https://arxiv.org/abs/2105.13120>, Jan. 2025.
- [92] Jia X, Song S, He W *et al.* Highly scalable deep learning training system with mixed-precision: Training ImageNet in four minutes. arXiv: 1807.11205, 2018. <https://arxiv.org/abs/1807.11205>, Nov. 2024.
- [93] Xu Y, Lee H, Chen D, Choi H, Hechtman B, Wang S. Automatic cross-replica sharding of weight update in data-parallel training. arXiv: 2004.13336, 2020. <https://arxiv.org/abs/2004.13336>, Nov. 2024.
- [94] Xu Q, You Y. An efficient 2D method for training super-large deep learning models. In *Proc. the 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2023, pp.222–232. DOI: [10.1109/IPDPS54959.2023.00031](https://doi.org/10.1109/IPDPS54959.2023.00031).
- [95] Wang B, Xu Q, Bian Z, You Y. Tesseract: Parallelize the tensor parallelism efficiently. In *Proc. the 51st International Conference on Parallel Processing*, Aug. 29–Sept. 1, 2022, Article No. 12. DOI: [10.1145/3545008.3545087](https://doi.org/10.1145/3545008.3545087).
- [96] Bian Z, Xu Q, Wang B, You Y. Maximizing parallelism in distributed training for huge neural networks. arXiv: 2105.14450, 2021. <https://arxiv.org/abs/2105.14450>, Nov. 2024.
- [97] Cheng S, Liu Z, Du J, You Y. ATP: Adaptive tensor parallelism for foundation models. arXiv: 2301.08658, 2023. <https://arxiv.org/abs/2301.08658>, Nov. 2024.
- [98] Zeng Z, Liu C, Tang Z, Li K, Li K. AccTFM: An effective intra-layer model parallelization strategy for training large-scale transformer-based models. *IEEE Trans. Parallel and Distributed Systems*, 2022, 33(12): 4326–4338. DOI: [10.1109/TPDS.2022.3187815](https://doi.org/10.1109/TPDS.2022.3187815).
- [99] Wang S, Wei J, Sabne A, Davis A, Ilbeyi B, Hechtman B, Chen D, Murthy K S, Maggioni M, Zhang Q, Kumar S, Guo T, Xu Y, Zhou Z. Overlap communication with dependent computation via decomposition in large deep learning models. In *Proc. the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2022, pp.93–106. DOI: [10.1145/3567955.3567959](https://doi.org/10.1145/3567955.3567959).
- [100] Huang Y, Cheng Y, Bapna A, Firat O, Chen M X, Chen D, Lee H, Ngiam J, Le Q V, Wu Y, Chen Z. GPipe: Efficient training of giant neural networks using pipeline parallelism. In *Proc. the 33rd International Conference on Neural Information Processing Systems*, Dec. 2019, Article No. 10.
- [101] Fan S, Rong Y, Meng C, Cao Z, Wang S, Zheng Z, Wu C, Long G, Yang J, Xia L, Diao L, Liu X, Lin W. DAPPLE: A pipelined data parallel approach for training large models. In *Proc. the 26th ACM SIGPLAN Symposium*

- sium on Principles and Practice of Parallel Programming, Feb. 2021, pp.431–445. DOI: [10.1145/3437801.3441593](https://doi.org/10.1145/3437801.3441593).
- [102] Narayanan D, Harlap A, Phanishayee A, Seshadri V, Devanur N R, Ganger G R, Gibbons P B, Zaharia M. PipeDream: Generalized pipeline parallelism for DNN training. In *Proc. the 27th ACM Symposium on Operating Systems Principles*, Oct. 2019, pp.1–15. DOI: [10.1145/3341301.3359646](https://doi.org/10.1145/3341301.3359646).
- [103] Narayanan D, Phanishayee A, Shi K, Chen X, Zaharia M. Memory-efficient pipeline-parallel DNN training. In *Proc. the 38th International Conference on Machine Learning*, Jul. 2021, pp.7937–7947.
- [104] Park J H, Yun G, Yi C M, Nguyen N T, Lee S, Choi J, Noh S H, Choi Y R. HetPipe: Enabling large DNN training on (Whimpy) heterogeneous GPU clusters through integration of pipelined model parallelism and data parallelism. In *Proc. the 2020 USENIX Annual Technical Conference*, Jul. 2020, pp.307–321.
- [105] Li S, Hoeftler T. Chimera: Efficiently training large-scale neural networks with bidirectional pipelines. In *Proc. the 2021 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2021, pp.1–14. DOI: [10.1145/3458817.3476145](https://doi.org/10.1145/3458817.3476145).
- [106] Jain A, Awan A A, Aljuhani A M, Hashmi J M, Anthony Q G, Subramoni H, Panda D K, Machiraju R, Parwani A. GEMS: GPU-enabled memory-aware model-parallelism system for distributed DNN training. In *Proc. the 2020 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2020, pp.1–15. DOI: [10.1109/SC41405.2020.00049](https://doi.org/10.1109/SC41405.2020.00049).
- [107] Kim T, Kim H, Yu G I, Chun B G. BPipe: Memory-balanced pipeline parallelism for training large language models. In *Proc. the 40th International Conference on Machine Learning*, Jul. 2023, pp.16639–16653.
- [108] Zhao S, Li F, Chen X, Guan X, Jiang J, Huang D, Qing Y, Wang S, Wang P, Zhang G, Li C, Luo P, Cui H. vPipe: A virtualized acceleration system for achieving efficient and scalable pipeline parallel DNN training. *IEEE Trans. Parallel and Distributed Systems*, 2022, 33(3): 489–506. DOI: [10.1109/TPDS.2021.3094364](https://doi.org/10.1109/TPDS.2021.3094364).
- [109] Tanaka M, Taura K, Hanawa T, Torisawa K. Automatic graph partitioning for very large-scale deep learning. In *Proc. the 2021 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2021, pp.1004–1013. DOI: [10.1109/IPDPS49936.2021.00109](https://doi.org/10.1109/IPDPS49936.2021.00109).
- [110] Miao X, Wang Y, Jiang Y, Shi C, Nie X, Zhang H, Cui B. Galvatron: Efficient transformer training over multiple GPUs using automatic parallelism. *Proc. the VLDB Endowment*, 2022, 16(3): 470–479. DOI: [10.14778/3570690.3570697](https://doi.org/10.14778/3570690.3570697).
- [111] Zheng L, Li Z, Zhang H, Zhuang Y, Chen Z, Huang Y, Wang Y, Xu Y, Zhuo D, Xing E P, Gonzalez J E, Stolica I. Alpa: Automating inter-and intra-operator parallelism for distributed deep learning. In *Proc. the 16th USENIX Symposium on Operating Systems Design and Implementation*, Jul. 2022, pp.559–578.
- [112] Luo Z, Yi X, Long G, Fan S, Wu C, Yang J, Lin W. Efficient pipeline planning for expedited distributed DNN training. In *Proc. the 2022 IEEE Conference on Computer Communications*, May 2022, pp.340–349. DOI: [10.1109/INFOCOM48880.2022.9796787](https://doi.org/10.1109/INFOCOM48880.2022.9796787).
- [113] Tarnawski J, Narayanan D, Phanishayee A. Piper: Multidimensional planner for DNN parallelization. In *Proc. the 35th International Conference on Neural Information Processing Systems*, Dec. 2021, pp.24829–24840.
- [114] Wang S, Rong Y, Fan S, Zheng Z, Diao L, Long G, Yang J, Liu X, Lin W. Auto-MAP: A DQN framework for exploring distributed execution plans for DNN workloads. arXiv: 2007.04069, 2020. <https://arxiv.org/abs/2007.04069>, Nov. 2024.
- [115] Jayaram Subramanya S, Arfeen D, Lin S, Qiao A, Jia Z, Ganger G R. Sia: Heterogeneity-aware, goodput-optimized ML-cluster scheduling. In *Proc. the 29th Symposium on Operating Systems Principles*, Oct. 2023, pp.642–657. DOI: [10.1145/3600006.3613175](https://doi.org/10.1145/3600006.3613175).
- [116] Jangda A, Huang J, Liu G, Sabet A H N, Maleki S, Miao Y, Musuvathi M, Mytkowicz T, Saarikivi O. Breaking the computation and communication abstraction barrier in distributed machine learning workloads. In *Proc. the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Feb. 28-Mar. 4, 2022, pp.402–416. DOI: [10.1145/3503222.3507778](https://doi.org/10.1145/3503222.3507778).
- [117] Chen C, Li X, Zhu Q, Duan J, Sun P, Zhang X, Yang C. Centauri: Enabling efficient scheduling for communication-computation overlap in large model training via communication partitioning. In *Proc. the 29th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Apr. 27-May 1, 2024, pp.178–191. DOI: [10.1145/3620666.3651379](https://doi.org/10.1145/3620666.3651379).
- [118] Nicolae B, Li J, Wozniak J M, Bosilca G, Dorier M, Cappello F. DeepFreeze: Towards scalable asynchronous checkpointing of deep learning models. In *Proc. the 20th IEEE/ACM International Symposium on Cluster, Cloud and Internet Computing (CCGRID)*, May 2020, pp.172–181. DOI: [10.1109/CCGrid49817.2020.00-76](https://doi.org/10.1109/CCGrid49817.2020.00-76).
- [119] Mohan J, Phanishayee A, Chidambaram V. CheckFreq: Frequent, fine-grained DNN checkpointing. In *Proc. the 19th USENIX Conference on File and Storage Technologies*, Feb. 2021, pp.203–216.
- [120] Eisenman A, Matam K K, Ingram S, Mudigere D, Krishnamoorthi R, Nair K, Smelyanskiy M, Annavaram M. Check-N-Run: A checkpointing system for training deep learning recommendation models. In *Proc. the 19th USENIX Symposium on Networked Systems Design and Implementation*, Apr. 2022, pp.929–943.
- [121] Wang Z, Jia Z, Zheng S, Zhang Z, Fu X, Ng T S E, Wang Y. GEMINI: Fast failure recovery in distributed training with in-memory checkpoints. In *Proc. the 29th Symposium on Operating Systems Principles*, Oct. 2023, pp.364–381. DOI: [10.1145/3600006.3613145](https://doi.org/10.1145/3600006.3613145).
- [122] Wu B, Xia L, Li Q, Li K, Chen X, Guo Y, Xiang T, Chen Y, Li S. TRANSOM: An efficient fault-tolerant system for training LLMs. arXiv: 2310.10046, 2023.

- <https://arxiv.org/abs/2310.10046>, Nov. 2024.
- [123] He T, Li X, Wang Z, Qian K, Xu J, Yu W, Zhou J. Unicorn: Economizing self-healing LLM training at scale. arXiv: 2401.00134, 2023. <https://arxiv.org/abs/2401.00134>, Nov. 2024.
- [124] Jiang Y, Zhu Y, Lan C, Yi B, Cui Y, Guo C. A unified architecture for accelerating distributed DNN training in heterogeneous GPU/CPU clusters. In *Proc. the 14th USENIX Symposium on Operating Systems Design and Implementation*, Nov. 2020, pp.463–479.
- [125] Luo Q, He J, Zhuo Y, Qian X. Prague: High-performance heterogeneity-aware asynchronous decentralized training. In *Proc. the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2020, pp.401–416. DOI: [10.1145/3373376.3378499](https://doi.org/10.1145/3373376.3378499).
- [126] Dong J, Cao Z, Zhang T et al. EFLOPS: Algorithm and system co-design for a high performance distributed training platform. In *Proc. the 2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, Feb. 2020, pp.610–622. DOI: [10.1109/HP-CA47549.2020.00056](https://doi.org/10.1109/HP-CA47549.2020.00056).
- [127] Zhao S, Li F, Chen X, Shen T, Chen L, Wang S, Zhang N, Li C, Cui H. NASPipe: High performance and reproducible pipeline parallel supernet training via causal synchronous parallelism. In *Proc. the 27th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Feb. 28–Mar. 4, 2022, pp.374–387. DOI: [10.1145/3503222.3507735](https://doi.org/10.1145/3503222.3507735).
- [128] Amaral M, Polo J, Carrera D, Seelam S, Steinder M. Topology-aware GPU scheduling for learning workloads in cloud environments. In *Proc. the 2017 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2017, Article No. 17. DOI: [10.1145/3126908.3126933](https://doi.org/10.1145/3126908.3126933).
- [129] Chen T, Xu B, Zhang C, Guestrin C. Training deep nets with sublinear memory cost. arXiv: 1604.06174, 2016. <https://arxiv.org/abs/1604.06174>, Nov. 2024.
- [130] Kusumoto M, Inoue T, Watanabe G, Akiba T, Koyama M. A graph theoretic framework of recomputation algorithms for memory-efficient backpropagation. In *Proc. the 33rd International Conference on Neural Information Processing Systems*, Dec. 2019, Article No. 105.
- [131] Jain P, Jain A, Nrusimha A, Gholami A, Abbeel P, Gonzalez J, Keutzer K, Stoica I. Checkmate: Breaking the memory wall with optimal tensor rematerialization. In *Proc. the 3rd Conference on Machine Learning and Systems*, Mar. 2020, pp.497–511.
- [132] Kirisame M, Lyubomirsky S, Haan A, Brennan J, He M, Roesch J, Chen T, Tatlock Z. Dynamic tensor rematerialization. In *Proc. the 9th International Conference on Learning Representations*, May 2021.
- [133] Korthikanti V A, Casper J, Lym S, McAfee L, Andersch M, Shoeybi M, Catanzaro B. Reducing activation recomputation in large transformer models. In *Proc. the 6th Conference on Machine Learning and Systems*, Jun. 2023.
- [134] Rhu M, Gimelshein N, Clemons J, Zulfiqar A, Keckler S W. vDNN: Virtualized deep neural networks for scalable, memory-efficient neural network design. In *Proc. the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 2016, pp.1–13. DOI: [10.1109/MICRO.2016.7783721](https://doi.org/10.1109/MICRO.2016.7783721).
- [135] Ren J, Rajbhandari S, Aminabadi R Y, Ruwase O, Yang S, Zhang M, Li D, He Y. ZeRO-offload: Democratizing billion-scale model training. In *Proc. the 2021 USENIX Annual Technical Conference*, Jul. 2021, pp.551–564.
- [136] Rajbhandari S, Ruwase O, Rasley J, Smith S, He Y. ZeRO-infinity: Breaking the GPU memory wall for extreme scale deep learning. In *Proc. the 2021 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2021. DOI: [10.1145/3458817.3476205](https://doi.org/10.1145/3458817.3476205).
- [137] Fang J R, Zhu Z L, Li S G, Su H, Yu Y, Zhou J, You Y. Parallel training of pre-trained models via chunk-based dynamic memory management. *IEEE Trans. Parallel and Distributed Systems*, 2023, 34(1): 304–315. DOI: [10.1109/TPDS.2022.3219819](https://doi.org/10.1109/TPDS.2022.3219819).
- [138] Zhang H, Zhou Y, E, Xue Y, Liu Y, Huang J. G10: Enabling an efficient unified GPU memory and storage architecture with smart tensor migrations. In *Proc. the 56th IEEE/ACM International Symposium on Microarchitecture (MICRO)*, Oct. 28–Nov. 1, 2023, pp.395–410.
- [139] Peng X, Shi X, Dai H, Jin H, Ma W, Xiong Q, Yang F, Qian X. Capuchin: Tensor-based GPU memory management for deep learning. In *Proc. the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2020, pp.891–905. DOI: [10.1145/3373376.3378505](https://doi.org/10.1145/3373376.3378505).
- [140] Wahib M, Zhang H, Nguyen T T, Drozd A, Domke J, Zhang L, Takano R, Matsuoka S. Scaling distributed deep learning workloads beyond the memory capacity with KARMA. In *Proc. the 2020 International Conference for High Performance Computing, Networking, Storage and Analysis*, Nov. 2020. DOI: [10.1109/SC41405.2020.00023](https://doi.org/10.1109/SC41405.2020.00023).
- [141] Zong Z, Lin L, Lin L, Wen L, Sun Y. STR: Hybrid tensor re-generation to break memory wall for DNN training. *IEEE Trans. Parallel and Distributed Systems*, 2023, 34(8): 2403–2418. DOI: [10.1109/TPDS.2023.3266110](https://doi.org/10.1109/TPDS.2023.3266110).
- [142] Huang C C, Jin G, Li J. SwapAdvisor: Pushing deep learning beyond the GPU memory limit via smart swapping. In *Proc. the 25th International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2020, pp.1341–1355. DOI: [10.1145/3373376.3378530](https://doi.org/10.1145/3373376.3378530).
- [143] He S, Chen P, Chen S, Li Z, Yang S, Chen W, Shou L. HOME: A holistic GPU memory management framework for deep learning. *IEEE Trans. Computers*, 2022, 72(3): 826–838. DOI: [10.1109/TC.2022.3180991](https://doi.org/10.1109/TC.2022.3180991).
- [144] Wang L, Ye J, Zhao Y, Wu W, Li A, Song S L, Xu Z, Kraska T. Superneurons: Dynamic GPU memory management for training deep neural networks. In *Proc. the 23rd ACM SIGPLAN Symposium on Principles and Practice of Parallel Programming*, Feb. 2018, pp.41–53. DOI: [10.1145/3178487.3178491](https://doi.org/10.1145/3178487.3178491).

- [145] Cambier L, Bhiwandiwala A, Gong T, Elibol O H, Nekui M, Tang H. Shifted and squeezed 8-bit floating point format for low-precision training of deep neural networks. In *Proc. the 8th International Conference on Learning Representations*, Apr. 2020.
- [146] Zhang X, Liu S, Zhang R, Liu C, Huang D, Zhou S, Guo J, Guo Q, Du Z, Zhi T, Chen Y. Fixed-point back-propagation training. In *Proc. the 2020 IEEE/CVF conference on computer vision and pattern recognition (CVPR)*, Jun. 2020, pp.2327–2335. DOI: [10.1109/CVPR42600.2020.00240](https://doi.org/10.1109/CVPR42600.2020.00240).
- [147] Peng H, Wu K, Wei Y et al. FP8-LM: Training FP8 large language models. arXiv: 2310.18313, 2023. <https://arxiv.org/abs/2310.18313>, Nov. 2024.
- [148] Dettmers T, Pagnoni A, Holtzman A, Zettlemoyer L. QLoRA: Efficient finetuning of quantized LLMs. In *Proc. the 37th International Conference on Neural Information Processing Systems*, Dec. 2023, Article No. 441.
- [149] Pan Z, Chen P, He H, Liu J, Cai J, Zhuang B. Mesa: A memory-saving training framework for transformers. arXiv: 2111.11124, 2021. <https://arxiv.org/abs/2111.11124>, Nov. 2024.
- [150] Liu X, Zheng L, Wang D, Cen Y, Chen W, Han X, Chen J, Liu Z, Tang J, Gonzalez J, Mahoney M W, Cheung A. GACT: Activation compressed training for generic network architectures. In *Proc. the 39th International Conference on Machine Learning*, Jul. 2022, pp.14139–14152.
- [151] Seide F, Fu H, Droppo J, Li G, Yu D. 1-bit stochastic gradient descent and its application to data-parallel distributed training of speech DNNs. In *Proc. the 15th Annual Conference of the International Speech Communication Association*, Sept. 2014, pp.1058–1062.
- [152] Tang H, Gan S, Awan A A, Rajbhandari S, Li C, Lian X, Liu J, Zhang C, He Y. 1-bit Adam: Communication efficient large-scale training with Adam’s convergence speed. In *Proc. the 38th International Conference on Machine Learning*, Jul. 2021, pp.10118–10129.
- [153] Lu Y, Li C, Zhang M, De Sa C, He Y. Maximizing communication efficiency for large-scale training via 0/1 Adam. In *Proc. the 11th International Conference on Learning Representations*, May 2023.
- [154] Jia Z, Padon O, Thomas J, Warszawski T, Zaharia M, Aiken A. TASO: Optimizing deep learning computation with automatic generation of graph substitutions. In *Proc. the 27th ACM Symposium on Operating Systems Principles*, Oct. 2019, pp.47–62. DOI: [10.1145/3341301.3359630](https://doi.org/10.1145/3341301.3359630).
- [155] Ivanov A, Dryden N, Ben-Nun T, Li S, Hoefer T. Data movement is all you need: A case study on optimizing transformers. In *Proc. the 4th Conference on Machine Learning and Systems*, Apr. 2021, pp.711–732.
- [156] Rabe M N, Staats C. Self-attention does not need $O(n^2)$ memory. arXiv: 2112.05682, 2021. <https://arxiv.org/abs/2112.05682>, Nov. 2024.
- [157] Dao T, Fu D Y, Ermon S, Rudra A, Ré C. FLASHATTENTION: Fast and memory-efficient exact attention with IO-awareness. In *Proc. the 36th International Conference on Neural Information Processing Systems*, Nov. 28–Dec. 9, 2022, Article No. 1189.
- [158] Dao T. FlashAttention-2: Faster attention with better parallelism and work partitioning. In *Proc. the 12th International Conference on Learning Representations*, May 2024.
- [159] Zheng S, Chen S, Song P, Chen R, Li X, Yan S, Lin D, Leng J, Liang Y. Chimera: An analytical optimizing framework for effective compute-intensive operators fusion. In *Proc. the 2023 IEEE International Symposium on High-Performance Computer Architecture (HPCA)*, Feb. 25–Mar. 1, 2023, pp.1113–1126. DOI: [10.1109/HPCA56546.2023.10071018](https://doi.org/10.1109/HPCA56546.2023.10071018).
- [160] Wang F, Shen M. Automatic kernel generation for large language models on deep learning accelerators. In *Proc. the 2023 IEEE/ACM International Conference on Computer Aided Design (ICCAD)*, Oct. 29–Nov. 2, 2023, pp.1–9. DOI: [10.1109/ICCAD57390.2023.10323944](https://doi.org/10.1109/ICCAD57390.2023.10323944).
- [161] Yang Y, Deng L, Wu S, Yan T, Xie Y, Li G. Training high-performance and large-scale deep neural networks with full 8-bit integers. *Neural Networks*, 2020, 125: 70–82. DOI: [10.1016/j.neunet.2019.12.027](https://doi.org/10.1016/j.neunet.2019.12.027).
- [162] Liu C, Zhang X, Zhang R, Li L, Zhou S, Huang D, Li Z, Du Z, Liu S, Chen T. Rethinking the importance of quantization bias, toward full low-bit training. *IEEE Trans. Image Processing*, 2022, 31: 7006–7019. DOI: [10.1109/TIP.2022.3216776](https://doi.org/10.1109/TIP.2022.3216776).
- [163] Zhao Y, Liu C, Du Z, Guo Q, Hu X, Zhuang Y, Zhang Z, Song X, Li W, Zhang X, Li L, Xu Z, Chen T. Cambricon-Q: A hybrid architecture for efficient training. In *Proc. the 48th ACM/IEEE Annual International Symposium on Computer Architecture (ISCA)*, Jun. 2021, pp.706–719. DOI: [10.1109/ISCA52012.2021.00061](https://doi.org/10.1109/ISCA52012.2021.00061).
- [164] Shi Y, Yang Z, Xue J, Ma L, Xia Y, Miao Z, Guo Y, Yang F, Zhou L. Welder: Scheduling deep learning memory access via tile-graph. In *Proc. the 17th USENIX Symposium on Operating Systems Design and Implementation*, Jul. 2023, pp.701–718.
- [165] Feng S, Hou B, Jin H, Lin W, Shao J, Lai R, Ye Z, Zheng L, Yu C H, Yu Y, Chen T. TensorIR: An abstraction for automatic tensorized program optimization. In *Proc. the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2023, pp.804–817. DOI: [10.1145/3575693.3576933](https://doi.org/10.1145/3575693.3576933).
- [166] Bi J, Guo Q, Li X, Zhao Y, Wen Y, Guo Y, Zhou E, Hu X, Du Z, Li L, Chen H, Chen T. Heron: Automatically constrained high-performance library generation for deep learning accelerators. In *Proc. the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems*, Mar. 2023, pp.314–328. DOI: [10.1145/3582016.3582061](https://doi.org/10.1145/3582016.3582061).
- [167] Kim S, Gholami A, Yao Z, Mahoney M W, Keutzer K. I-BERT: Integer-only BERT quantization. In *Proc. the 38th International Conference on Machine Learning*, Jul. 2021, pp.5506–5518.
- [168] Kwon W, Li Z, Zhuang S, Sheng Y, Zheng L, Yu C H, Gonzalez J, Zhang H, Stoica I. Efficient memory man-

agement for large language model serving with PagedAttention. In *Proc. the 29th Symposium on Operating Systems Principles*, Oct. 2023, pp.611–626. DOI: [10.1145/3600006.3613165](https://doi.org/10.1145/3600006.3613165).

- [169] Liu Z, Desai A, Liao F, Wang W, Xie V, Xu Z, Kyrillidis A, Shrivastava A. Scissorhands: Exploiting the persistence of importance hypothesis for LLM KV cache compression at test time. In *Proc. the 37th International Conference on Neural Information Processing Systems*, Dec. 2023, Article No. 2279.
- [170] Ge S, Zhang Y, Liu L, Zhang M, Han J, Gao J. Model tells you what to discard: Adaptive KV cache compression for LLMs. In *Proc. the 12th International Conference on Learning Representations*, May 2024.
- [171] Zhai Y, Jiang C, Wang L, Jia X, Zhang S, Chen Z, Liu X, Zhu Y. ByteTransformer: A high-performance transformer boosted for variable-length inputs. In *Proc. the 2023 IEEE International Parallel and Distributed Processing Symposium (IPDPS)*, May 2023, pp.344–355. DOI: [10.1109/IPDPS54959.2023.00042](https://doi.org/10.1109/IPDPS54959.2023.00042).
- [172] Yu G I, Jeong J S, Kim G W, Kim S, Chun B G. Orca: A distributed serving system for transformer-based generative models. In *Proc. the 16th USENIX Symposium on Operating Systems Design and Implementation*, Jul. 2022, pp.521–538.
- [173] Agrawal A, Panwar A, Mohan J, Kwatra N, Gulavani B S, Ramjee R. SARATHI: Efficient LLM inference by piggybacking decodes with chunked prefills. arXiv: 2308.16369, 2023. <https://arxiv.org/abs/2308.16369>, Nov. 2024.
- [174] Shen H, Roesch J, Chen Z, Chen W, Wu Y, Li M, Sharma V, Tatlock Z, Wang Y. Nimble: Efficiently compiling dynamic neural networks for model inference. In *Proc. the Fourth Conference on Machine Learning and Systems*, Apr. 2021, pp.208–222.
- [175] Fegade P, Chen T, Gibbons P, Mowry T. The CoRa tensor compiler: Compilation for ragged tensors with minimal padding. In *Proc. the 5th Conference on Machine Learning and Systems*, Aug. 2022, pp.721–747.
- [176] Sun Y, Dong L, Huang S, Ma S, Xia Y, Xue J, Wang J, Wei F. Retentive network: A successor to transformer for large language models. arXiv: 2307.08621, 2023. <https://arxiv.org/abs/2307.08621>, Nov. 2024.
- [177] Yu L, Simig D, Flaherty C, Aghajanyan A, Zettlemoyer L, Lewis M. MEGABYTE: Modeling million-byte sequences with multiscale transformers. In *Proc. the 37th International Conference on Neural Information Processing Systems*, Dec. 2023, Article No. 3447.
- [178] Dao T, Gu A. Transformers are SSMS: Generalized models and efficient algorithms through structured state space duality. In *Proc. the 41st International Conference on Machine Learning*, Jul. 2024.
- [179] Zhao L, Maleki S, Shah A, Yang Z, Pourreza H, Krishnamurthy A. ForestColl: Efficient collective communications on heterogeneous network fabrics. arXiv: 2402.06787, 2024. <https://arxiv.org/abs/2402.06787>, Nov. 2024.



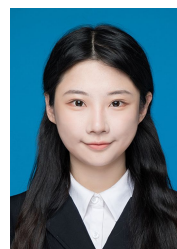
Zhen-Xing Zhang received his B.S. degree in information and computational science from the University of Science and Technology of China, Hefei, in 2017, where he is currently pursuing his Ph.D. degree. He is also a visiting student at the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing. His current research interests include computer architecture and parallel computing.



Yuan-Bo Wen received his B.E. and Ph.D. degrees from the University of Science and Technology of China, Hefei, in 2017, and 2022, respectively. He is working as an assistant professor at the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing. His current research interests include computer architecture and deep learning compiler.



Han-Qi Lyu received his B.E. degree in computer science from University of Science and Technology of China (USTC), Hefei, in 2023. He is currently a master student at USTC, Hefei. He is also a visiting student at the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing. His research interests include deep learning and reinforcement learning.



Chang Liu received her B.E. degree from Tsinghua University, Beijing, in 2018, and her Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, in 2023. She is currently an algorithm researcher at Cambricon Technologies, Beijing, where her research focuses on neural network quantization.



Rui Zhang received her Ph.D. degree in computer application technology from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, in 2019. She is working as an associate professor at the State Key Laboratory of Processors, ICT, CAS, Beijing. Her current research interests include deep learning, reinforcement learning, and computer vision.



Qi Guo received his B.E. degree in computer science from Tongji University, Shanghai, in 2007, and his Ph.D. degree from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, in 2012. He is currently a professor at ICT, CAS, Beijing. His research interests include computer architecture, machine learning, and programming models.



Xia-Qing Li received his Ph.D. degree from Tsinghua University, Beijing, in 2020. He is working as an associate professor at the Department of Computer Science and Technology, Beijing Jiaotong University, Beijing. His current research interests include parallel computing, high-performance systems of deep learning, and hyper-parameter tuning.



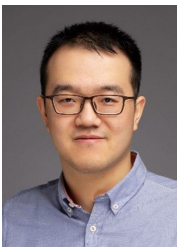
Ling Li received her B.S. degree from Wuhan University, Wuhan, in 2004, and her Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, in 2009. She is currently a professor at the Institute of Software, Chinese Academy of Sciences, Beijing. Her research interest is intelligent computing.



Chao Wang received his B.S. and Ph.D. degrees from the School of Computer Science, University of Science and Technology of China (USTC), Hefei, in 2006 and 2011, respectively. He is currently an associate professor at USTC, Hefei. His research interests focus on multicore and reconfigurable computing.



Xue-Hai Zhou received his B.S., M.S., and Ph.D. degrees from the University of Science and Technology of China (USTC), Hefei, in 1987, 1990, and 1997, respectively. He is a professor at the School of Computer Science of USTC, Hefei. His current research interests include various aspects of multicore and distributing systems.



Zi-Dong Du received his B.S. degree in engineering from the Department of Electronic Engineering, Tsinghua University, Beijing, in 2011, and his Ph.D. degree from the Institute of Computing Technology, Chinese Academy of Sciences, Beijing, in 2016. He is currently a professor at the State Key Laboratory of Processors, Institute of Computing Technology, Chinese Academy of Sciences, Beijing.



Yun-Ji Chen graduated from the Special Class for the Gifted Young, University of Science and Technology of China, Hefei, in 2002. He received his Ph.D. degree in computer science from the Institute of Computing Technology (ICT), Chinese Academy of Sciences (CAS), Beijing, in 2007. He is currently a professor at ICT, CAS, Beijing. His research interests include parallel computing, microarchitecture, hardware verification, and computational intelligence.